Module 1 – OVERVIEW OF THE WHOLE SYLLABUS

Data Types to Data Structures: Data Structures, Classifications: Primitive & Non-Primitive (Stack, Queue, Linked List, Graph, Trees, Hash Table). Data structure Operations.

Pointers & Functions in C - In-depth understanding of Pointers and Addresses, Pointers and Function arguments

Dynamic Memory Allocation Functions – Implementation with Programming Examples (Dynamic 1D array, 2D array), review of structures, Sparse Matrices

Text Books:

1. Ellis Horowitz and Sartaj Sahni, Fundamentals of Data Structures in C, 2nd Ed, Universities Press, 2014.
2. Brian W. Kernighan and Dennis M. Ritchie, The 'C' Programming Language, Prentice Hall of India.

An Introduction to Data Structures

Data is a collection of facts and figures or a set of values or values of a specific format that refers to a single set of item values. The data items are then classified into sub-items, which is the group of items that are not known as the simple primary form of the item.

Let us consider an example where an employee name can be broken down into three sub-items: First, Middle, and Last. However, an ID assigned to an employee will generally be considered a single item.

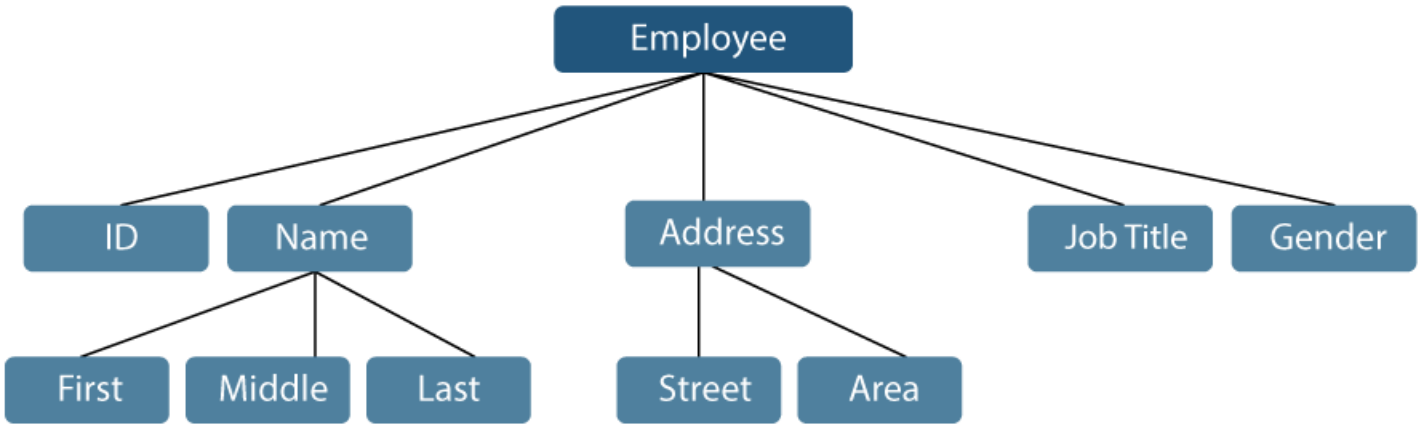


Fig. 1.1: Representation of Data Items

In Fig. 1.1 the example mentioned above, the items such as ID, Age, Gender, First, Middle, Last, Street, Locality, etc., are elementary data items. In contrast, the Name and the Address are group data items.

Data Structures is about how data can be stored in different structures.

If we want to store data about people we are related to, we can choose a family tree (Fig. 1.2) as the data structure. We have information about people we are related to and how they are related, and we want an overview so that we can easily find a specific family member, several generations back.

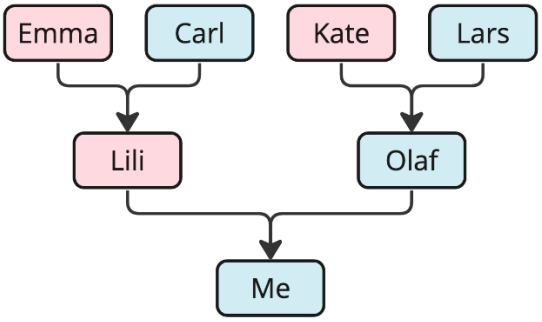


Fig. 1.2 Family tree

With such a family tree data structure visually in front of you, it is easy to see, for example, who my mother's mother is—it is 'Emma,' right? But without the links from child to parents that this data structure provides, it would be difficult to determine how the individuals are related.

Data structures give us the possibility to manage large amounts of data efficiently for uses such as large databases and internet indexing services.

Data structures are essential ingredients in creating fast and powerful algorithms. They help in managing and organizing data, reduce complexity, and increase efficiency.

What is Data Structure?

- **Data Structure** is a branch of Computer Science.
- **Data Structures** is about how data can be stored in different structures.
- The study of data structure allows us to understand the organization of data and the management of the data flow in order to increase the efficiency of any process or program.
- **Data Structure is a particular way of storing and organizing data in the memory of the computer so that these data can easily be retrieved and efficiently utilized in the future when required**.
- The data can be managed in various ways, like the logical or mathematical model for a specific organization of data is known as a data structure.

Some Applications of Data Structures

1. Data Structures help in the organization of data in a computer's memory.
2. Data Structures also help in representing the information in databases.
3. Data Structures allows the implementation of algorithms to search through data (For example, search engine).
4. We can use the Data Structures to implement the algorithms to manipulate data (For example, word processors).
5. We can also implement the algorithms to analyse data using Data Structures (For example, data miners).
6. Data Structures support algorithms to generate the data (For example, a random number generator).

7. Data Structures also support algorithms to compress and decompress the data (For example, a zip utility).
8. We can also use Data Structures to implement algorithms to encrypt and decrypt the data (For example, a security system).
9. With the help of Data Structures, we can build software that can manage files and directories (For example, a file manager).
10. We can also develop software that can render graphics using Data Structures. (For example, a web browser or 3D rendering software).
11. For managing large amounts of data, such as in a social network or a search engine.
12. For scheduling tasks, to decide which task a computer should do first.
13. For planning routes, like in a GPS system to find the shortest path from A to B.
14. For optimizing processes, such as arranging tasks so they can be completed as quickly as possible.
15. For solving complex problems: From finding the best way to pack a truck to making a computer 'learn' from data.

DSA is fundamental in nearly every part of the software world:

- Operating Systems
- Database Systems
- Web Applications
- Machine Learning
- Video Games
- Cryptographic Systems
- Data Analysis
- Search Engines
- Graphics
- Computer Design
- Blockchain
- Genetics
- Image Processing
- Simulation

Classification of Data Structures:

We can classify Data Structures into two categories:

1. Primitive Data Structure
2. Non-Primitive Data Structure

Fig. 1.3 shows the different classifications of Data Structures.

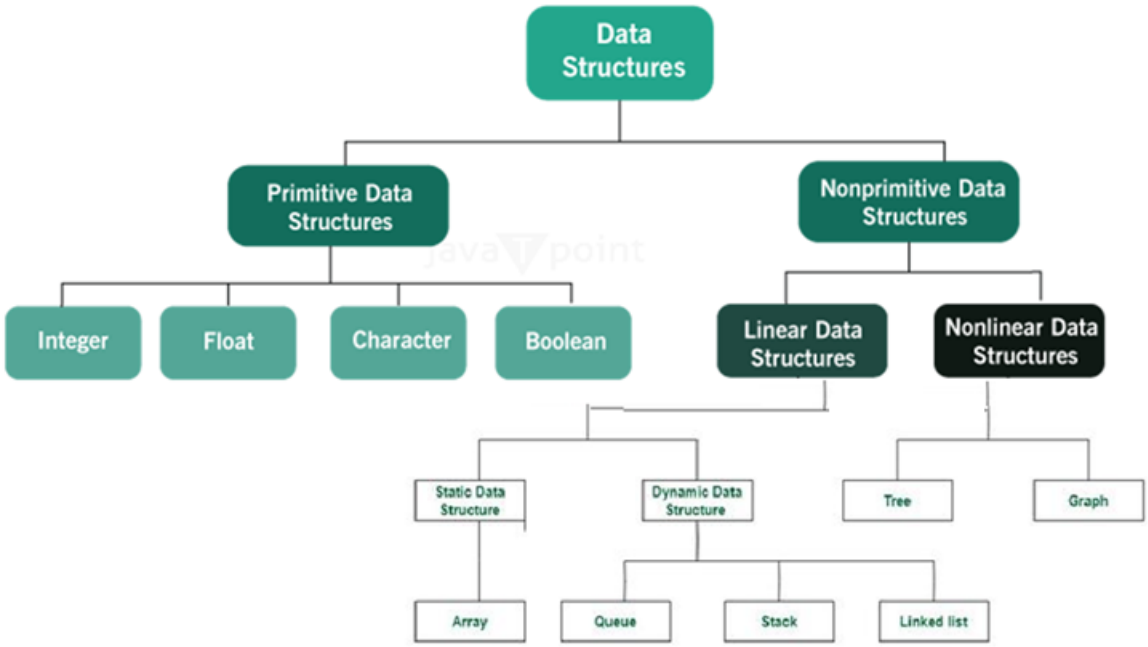


Fig.1.3: Classifications of Data Structures

Primitive Data Structures

1. **Primitive Data Structures** are the data structures consisting of the numbers and the characters that come **in-built** into programs.
2. These data structures can be manipulated or operated directly by machine-level instructions.
3. Basic data types like **Integer, Float, Character**, and **Boolean** come under the Primitive Data Structures.
4. These data types are also called **Simple data types**, as they contain characters that can't be divided further

Non-Primitive Data Structures

1. **Non-Primitive Data Structures** are those data structures derived from Primitive Data Structures.
2. These data structures can't be manipulated or operated directly by machine-level instructions.
3. The focus of these data structures is on forming a set of data elements that is either **homogeneous** (same data type) or **heterogeneous** (different data types).
4. Based on the structure and arrangement of data, we can divide these data structures into two sub-categories -
   a. Linear Data Structures
   b. Non-Linear Data Structures

Linear Data Structures

- A data structure that preserves a linear connection among its data elements is known as a **Linear Data Structure**.

- Elements are arranged in one dimension, also known as **linear dimension**.
- The arrangement of the data is done linearly, where each element consists of the successors and predecessors except the first and the last data element. However, it is not necessarily true in the case of memory, as the arrangement may not be sequential.
- Example: lists, stack, queue, etc.

Based on memory allocation, the Linear Data Structures are further classified into two types:

1. **Static Data Structures:** The data structures having a fixed size are known as Static Data Structures. The memory for these data structures is allocated at the compile time, and their size cannot be changed by the user after being compiled; however, the data stored in them can be altered. The **Array** is the best example of the Static Data Structure as they have a fixed size, and its data can be modified later.
2. **Dynamic Data Structures:** The data structures having a dynamic size are known as Dynamic Data Structures. The memory of these data structures is allocated at the run time, and their size varies during the run time of the code. Moreover, the user can change the size as well as the data elements stored in these data structures at the run time of the code.
   Linked Lists, Stacks, and **Queues** are common examples of dynamic data structures

Non-Linear Data Structure

- Non-Linear Data Structures are data structures where the data elements are not arranged in sequential order. Here, the insertion and removal of data are not feasible in a linear manner. There exists a hierarchical relationship between the individual data items.
- Elements are arranged in one-many, many-one and many-many dimensions.
- Example: tree, graph, table, etc.

1. Arrays

An Array is a data structure used to collect multiple data elements of the same data type into one variable. Instead of storing multiple values of the same data types in separate variable names, we could store all of them together into one variable.

The data elements of the array share the same **variable name**; however, each carries a different index number called a **subscript**. We can access any data element from the list with the help of its location in the list. Thus, the elements of an array are stored in contiguous memory locations, making it possible for the users to traverse through the data elements of the array using their respective indexes.

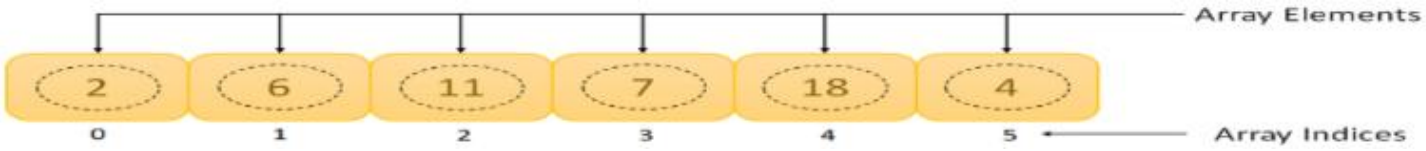


Fig. 1.4 An Array

Arrays can be classified into different types:

a) **One-Dimensional Array**: An Array with only one row of data elements is known as a One-Dimensional Array. It is stored in ascending storage location.

b) **Two-Dimensional Array**: An Array consisting of multiple rows and columns of data elements is called a Two-Dimensional Array. It is also known as a Matrix.

c) **Multidimensional Array**: We can define Multidimensional Array as an Array of Arrays. Multidimensional Arrays are not bounded to two indices or two dimensions as they can include as many indices are per the need.

Some Applications of Array:

a) We can store a list of data elements belonging to the same data type.

b) Array acts as an auxiliary storage for other data structures.

c) Array also acts as a storage of matrices.

2. Linked Lists

A **Linked List** is another example of a linear data structure used to store a collection of data elements dynamically. Data elements in this data structure are represented by the Nodes, connected using links or pointers. Each node contains two fields: the information field consists of the actual data, and the pointer field consists of the address of the subsequent nodes in the list. The pointer of the last node of the linked list consists of a null pointer, as it points to nothing. Unlike the Arrays, the user can dynamically adjust the size of a Linked List as per the requirements.

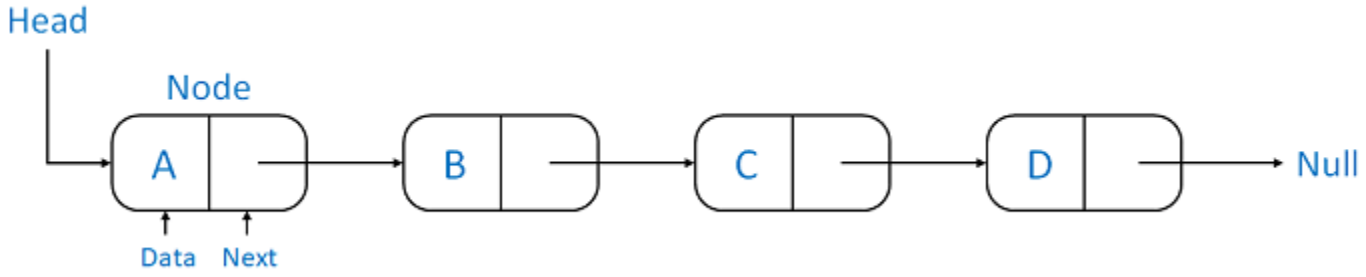


Fig.1.5 A Linked List

Linked Lists can be classified into different types:

a) Singly Linked List: A Singly Linked List is the most common type of Linked List. Each node has data and a pointer field containing an address to the next node.

b) Doubly Linked List: A Doubly Linked List consists of an information field and two pointer fields. The information field contains the data. The first pointer field contains an address of the previous node, whereas another pointer field contains a reference to the next node. Thus, we can go in both directions (backward as well as forward).

c) Circular Linked List: The Circular Linked List is similar to the Singly Linked List. The only key difference is that the last node contains the address of the first node, forming a circular loop in the Circular Linked List.

Some Applications of Linked Lists:

a. The Linked Lists help us implement stacks, queues, binary trees, and graphs of predefined size.

b. We can also implement Operating System's function for dynamic memory management.
c. Linked Lists also allow polynomial implementation for mathematical operations.
d. We can use Circular Linked List to implement Operating Systems or application functions that Round Robin execution of tasks.
e. Circular Linked List is also helpful in a Slide Show where a user requires to go back to the first slide after the last slide is presented.
f. Doubly Linked List is utilized to implement forward and backward buttons in a browser to move forward and backward in the opened pages of a website.

3. Stacks

- A **Stack** is a Linear Data Structure that follows the **LIFO** (Last In, First Out) principle that allows operations like insertion and deletion from one end of the Stack, i.e., Top.
- Stacks can be implemented with the help of contiguous memory, an Array, and non-contiguous memory, a Linked List.
- Real-life examples of Stacks are piles of books, a deck of cards, piles of money, and many more.

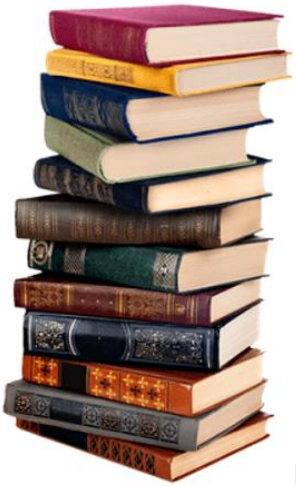


Fig. 1.6 A Real-life Example of Stack

Fig.1.6 represents the real-life example of a Stack where the operations are performed from one end only, like the insertion and removal of new books from the top of the Stack. It implies that the insertion and deletion in the Stack can be done only from the top of the Stack. We can access only the Stack's tops at any given time.

The primary operations in the Stack are as follows:

a. **Push:** Operation to insert a new element in the Stack is termed as Push Operation.
b. **Pop:** Operation to remove or delete elements from the Stack is termed as Pop Operation.

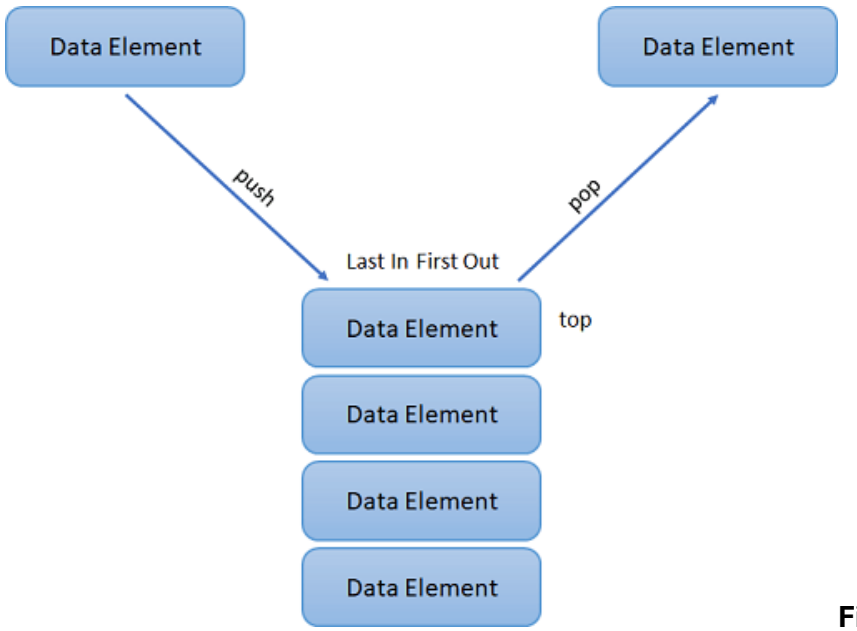


Fig.1.7 A Stack

Some Applications of Stacks:

a. The Stack is used as a Temporary Storage Structure for recursive operations.
b. Stack is also utilized as Auxiliary Storage Structure for function calls, nested operations, and deferred/postponed functions.
c. We can manage function calls using Stacks.
d. Stacks are also utilized to evaluate the arithmetic expressions in different programming languages.
e. Stacks are also helpful in converting infix expressions to postfix expressions.
f. Stacks allow us to check the expression's syntax in the programming environment.
g. We can match parenthesis using Stacks.
h. Stacks can be used to reverse a String.
i. Stacks are helpful in solving problems based on backtracking.
j. We can use Stacks in depth-first search in graph and tree traversal.
k. Stacks are also used in Operating System functions.
l. Stacks are also used in UNDO and REDO functions in an edit.

4. Queues

- A **Queue** is a linear data structure similar to a Stack with some limitations on the insertion and deletion of the elements.
- The insertion of an element in a Queue is done at one end, and the removal is done at another or opposite end.
- Thus, we can conclude that the Queue data structure follows FIFO (First In, First Out) principle to manipulate the data elements.
- Implementation of Queues can be done using Arrays, Linked Lists, or Stacks.
- Some real-life examples of Queues are a line at the ticket counter, an escalator, a car wash, and many more.

Fig.1.8 A Real-life Example of Queue

Fig. 1.8 is a real-life illustration of a movie ticket counter that can help us understand the Queue where the customer who comes first is always served first. The customer arriving last will undoubtedly be served last. Both ends of the Queue are open and can execute different operations. Another example is a food court line where the customer is inserted from the rear end while the customer is removed at the front end after providing the service they asked for.

The following are the primary operations of the Queue:

a. **Enqueue:** The insertion or Addition of some data elements to the Queue is called Enqueue. The element insertion is always done with the help of the rear pointer.

b. **Dequeue:** Deleting or removing data elements from the Queue is termed Dequeue. The deletion of the element is always done with the help of the front pointer.

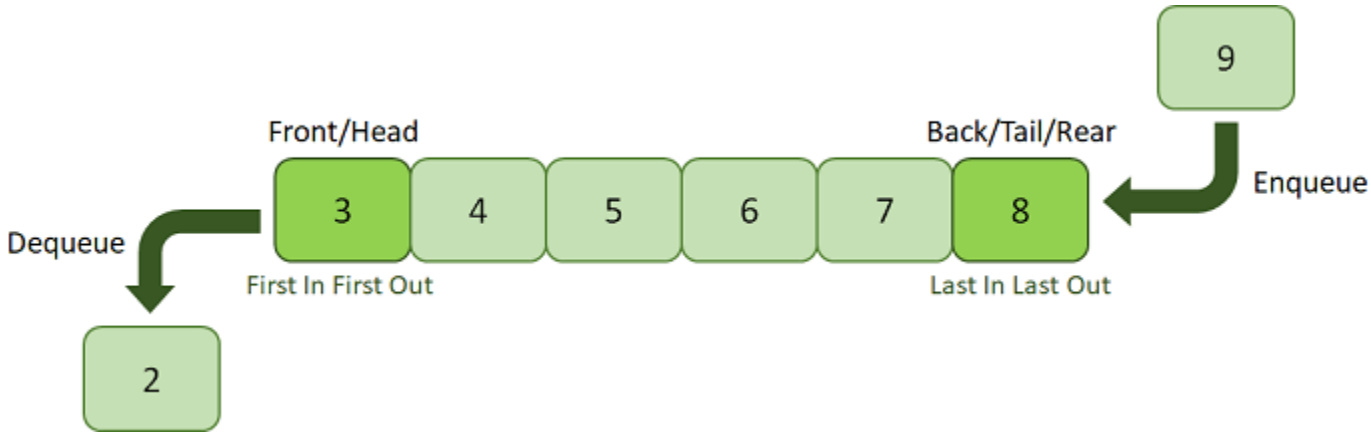
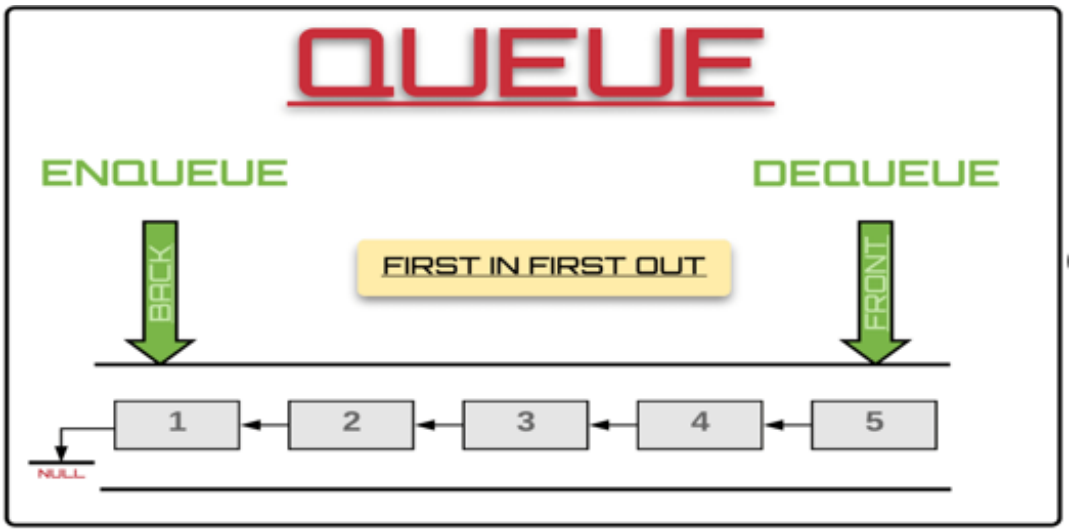


Figure 8. A Queue



Some Applications of Queues:

a. Queues are generally used in the breadth search operation in Graphs.

b. Queues are also used in Job Scheduler Operations of Operating Systems, like a keyboard buffer queue to store the keys pressed by users and a print buffer queue to store the documents printed by the printer.

c. Queues are responsible for CPU scheduling, Job scheduling, and Disk Scheduling.

d. Priority Queues are utilized in file-downloading operations in a browser.

e. Queues are also used to transfer data between peripheral devices and the CPU.

f. Queues are also responsible for handling interrupts generated by the User Applications for the CPU.

Non-Linear Data Structure

- Non-Linear Data Structures are data structures where the data elements are not arranged in sequential order. Here, the insertion and removal of data are not feasible in a linear manner. There exists a hierarchical relationship between the individual data items.
- Elements are arranged in one-many, many-one and many-many dimensions.
- Example: tree, graph, table, etc.

The following is the list of different types of Non-Linear Data Structures that we generally use:

4. Graphs

- A Graph is another example of a Non-Linear Data Structure comprising a finite number of nodes or vertices and the edges connecting them.
- The Graphs are utilized to address problems of the real world in which it denotes the problem area as a network such as social networks, circuit networks, and telephone networks. For instance, the nodes or vertices of a Graph can represent a single user in a telephone network, while the edges represent the link between them via telephone.
- The Graph data structure, G is considered a mathematical structure comprised of a set of vertices, V and a set of edges, E as shown below:
  G = (V,E)

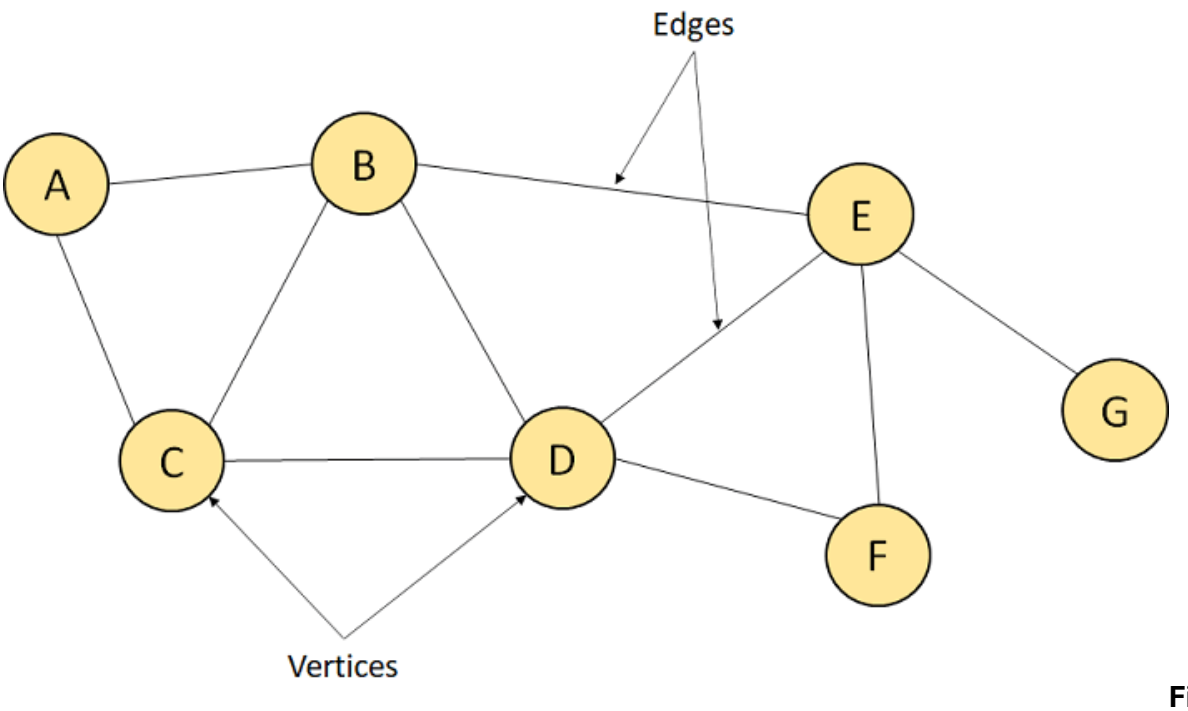


Figure. A Graph

The above figure represents a Graph having seven vertices A, B, C, D, E, F, G, and ten edges [A, B], [A, C], [B, C], [B, D], [B, E], [C, D], [D, E], [D, F], [E, F], and [E, G].

Some Applications of Graphs:

a. Graphs help us represent routes and networks in transportation, travel, and communication applications.
b. Graphs are used to display routes in GPS.
c. Graphs also help us represent the interconnections in social networks and other network-based applications.
d. Graphs are utilized in mapping applications.
e. Graphs are responsible for the representation of user preference in e-commerce applications.
f. Graphs are also used in Utility networks in order to identify the problems posed to local or municipal corporations.
g. Graphs also help to manage the utilization and availability of resources in an organization.
h. Graphs are also used to make document link maps of the websites in order to display the connectivity between the pages through hyperlinks.
i. Graphs are also used in robotic motions and neural networks.

5. Trees

- A Tree is a Non-Linear Data Structure and a hierarchy containing a collection of nodes such that each node of the tree stores a value and a list of references to other nodes (the "children").
- The Tree data structure is a specialized method to arrange and collect data in the computer to be utilized more effectively.
- It contains a central node, structural nodes, and sub-nodes connected via edges.
- We can also say that the tree data structure consists of roots, branches, and leaves connected.

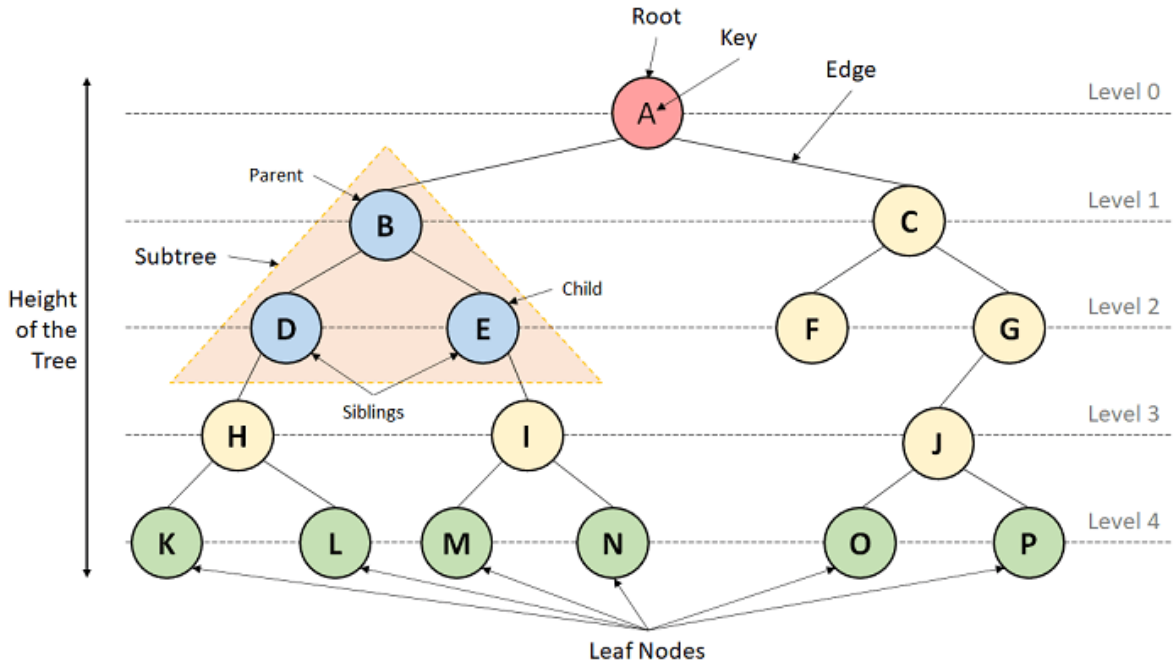


Figure. A Tree

Some Applications of Trees:

a. Trees implement hierarchical structures in computer systems like directories and file systems.

b. Trees are also used to implement the navigation structure of a website.

c. We can generate code like Huffman's code using Trees.

d. Trees are also helpful in decision-making in Gaming applications.

e. Trees are responsible for implementing priority queues for priority-based OS scheduling functions.

f. Trees are also responsible for parsing expressions and statements in the compilers of different programming languages.

g. We can use Trees to store data keys for indexing for Database Management System (DBMS).

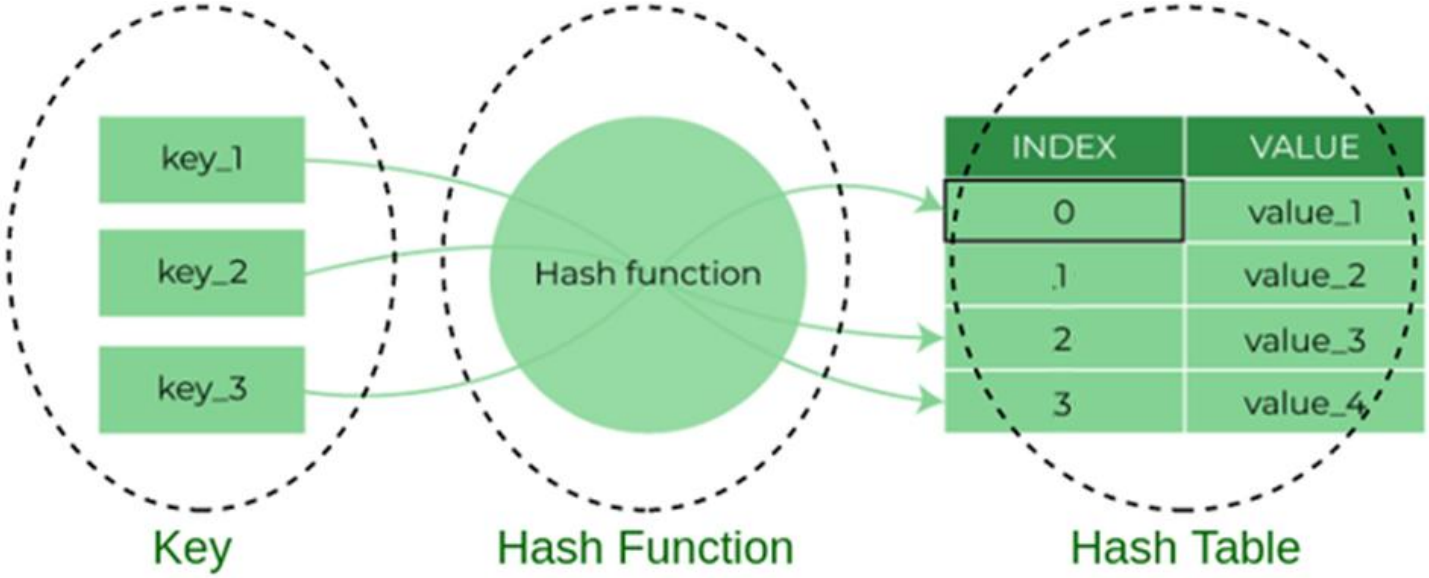
h. Spanning Trees allows us to route decisions in Computer and Communications Networks.

i. Trees are also used in the path-finding algorithm implemented in Artificial Intelligence (AI), Robotics, and Video Games Applications.

5. Hashing in Data Structure

- **Hashing** is a technique used in data structures that efficiently stores and retrieves data in a way that allows for quick access. It involves using a **hash function** to map data items to a fixed-size array which is called a **hash table**.
- The efficiency of mapping depends on the efficiency of the hash function used.
- Let a hash function H(x) maps the value x at the index x%10 in an Array.
- For example, if the list of values is [11, 12, 13, 14, 15] it will be stored at positions {1, 2, 3, 4, 5} in the array or Hash table respectively.

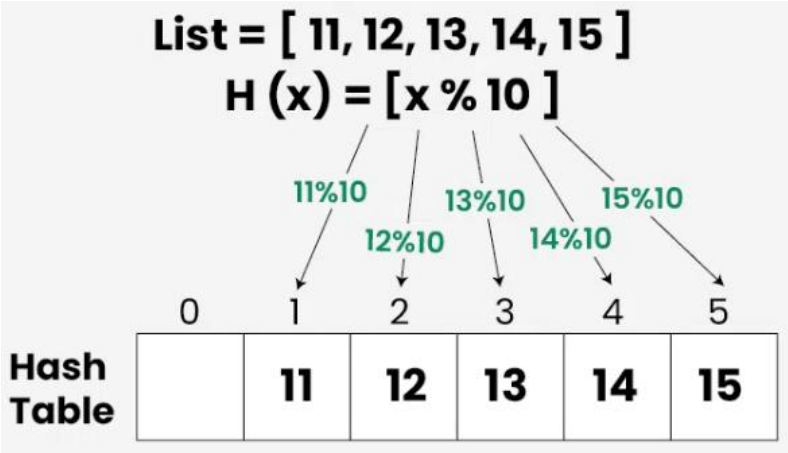
The hash function used here is “mod 10”.



Components of Hashing

Basic terminologies in hashing.

1. **Hash Function:** You provide your data items into the hash function.
2. **Hash Code:** The hash function crunches the data and give a unique hash code. This hash code is typically integer value that can be used an index.
3. **Hash Table:** The hash code then points you to a specific location within the hash table.



This method is commonly used in databases, **c**aching systems, and various programming applications to optimize search and retrieval operations. The great thing about hashing is, we can achieve all three operations (search, insert and delete) in O(1) time on average.

A **hash table** is also referred as a hash map (key value pairs) or a hash set (only keys). It uses a **hash function** to map **keys** to a fixed-size array, called a **hash table**. This allows in faster **search**, **insertion**, and **deletion** operations.

Hash Function

The **hash function** is a function that takes a **key** and returns an **index** into the **hash table**. The goal of a hash function is to distribute keys evenly across the hash table, minimizing collisions (when two keys map to the same index).

Hash tables are used wherever we have a combinations of search, insert and/or delete operations.

- **Dictionaries :** To implement a dictionary so that we can quickly search a word
- **Databases:** Hashing is used in database indexing. There are two popular ways to implement indexing, search trees (B or B+ Tree) and hashing.
- **Cryptography :** When we create a password on a website, they typically store it after applying a hash function rather than plain text
- **Caching:** Storing frequently accessed data for faster retrieval. For example browser caches, we can use URL as keys and find the local storage of the URL.
- **Symbol Tables:** Mapping identifiers to their values in programming languages
- **Network Routing:** Determining the best path for data packets

DIFFERENCE BETWEEN LINEAR AND NON LINEAR DATA STRUCTURES

Linear Data Structure

- Every item is related to its previous and next item.
- Data is arranged in linear sequence.
- Data items can be traversed in a single run
- E.g. Array, Stacks, Linked list, Queue
- Implementation is easy.

Non-Linear Data Structure

- Every item is attached with many other items.
- Data is not arranged in sequence.
- Data cannot be traversed in a single run.
- E.g. Tree, Graph
- Implementation is difficult.

Basic Operations of Data Structures

- The data appearing in our data structures are processed by means of certain operations.
- In fact, the particular data structure that one chooses for a given situation depends largely in the frequency with which specific operations are performed.
- The different operations that can be performed on the various data structures are as follows.

1. **Create:** Create is an operation used to reserve memory for the data elements of the program. We can perform this operation using a declaration statement. The creation of data structure can take place either during the following:
    a. Compile-time
    b. Run-time

    For example, the **malloc()** function is used in C Language to create data structure.
2. **Insertion:** Insertion means inserting or adding new data elements to the collection. For example, we can use the insertion operation to add the details of a new employee the company has recently hired.
3. **Deletion:** Deletion means to remove or delete a specific data element from the given list of data elements. For example, we can use the deleting operation to delete the name of an employee who has left the job.

4. **Destroy:** This operation destroys memory space allocated for specified data structure.
5. **Selection:** Selection means selecting a particular data from the available data. We can select any particular data by specifying conditions inside the loop.
6. **Update:** The Update operation allows us to update or modify the data in the data structure. We can also update any particular data by specifying some conditions inside the loop, like the Selection operation.
7. **Search:** Search is another data structure operation which means to find the location of one or more data elements that meet certain constraints. Such a data element may or may not be present in the given set of data elements. For example, we can use the search operation to find the names of all the employees who have the experience of more than 5 years.
8. **Sorting:** Sorting means to arrange the data elements in either Ascending or Descending order depending on the type of application. For example, we can use the sorting operation to arrange the names of employees in a department in alphabetical order or estimate the top three performers of the month by arranging the performance of the employees in descending order and extracting the details of the top three.
9. **Merge:** Merge means to combine data elements of two sorted lists in order to form a single list of sorted data elements.
10. **Splitting:** The Splitting operation allows us to divide data into various subparts decreasing the overall process completion time.
11. **Traversal:** Traversing a data structure means accessing each data element exactly once so it can be administered. For example, traversing is required while printing the names of all the employees in a department.

ARRAYS

Array Definition:

An **array** is a list of a finite number n of **homogeneous** data elements (i.e. data elements of the same type) such that:

i) The elements of the array are referenced respectively by an **index** set consisting of n consecutive numbers.
ii) The elements of the array are stored respectively in successive memory locations.

We assume that the index set consists of the integers 1, 2, ... , n, if not explicitly stated.

The number n of elements is called the *length* or *size* of the array. In general the length of the array can be obtained from the index set by using the formula:

Length = UB – LB +1 where UB is the upper bound i.e. the largest of the array,

LB is the lower bound i.e. the smallest of the array.

Suppose we have an array **A** with subscripts 1, 2, 3, ..., n, then the elements of **A** are denoted by

i) the subscript notation $\mathbf{A}_1, \mathbf{A}_2, \mathbf{A}_3 \ldots \mathbf{A}_n$ or
ii) the parenthesis notation A (1), A (2), A (3) . . . . . A (n) or

iii) the bracket notation A [1], A [2], A [3] . . . . . A [n]

The number K in A[K] is called a **subscript** or **index** and A[K] is called a **subscripted variable**.

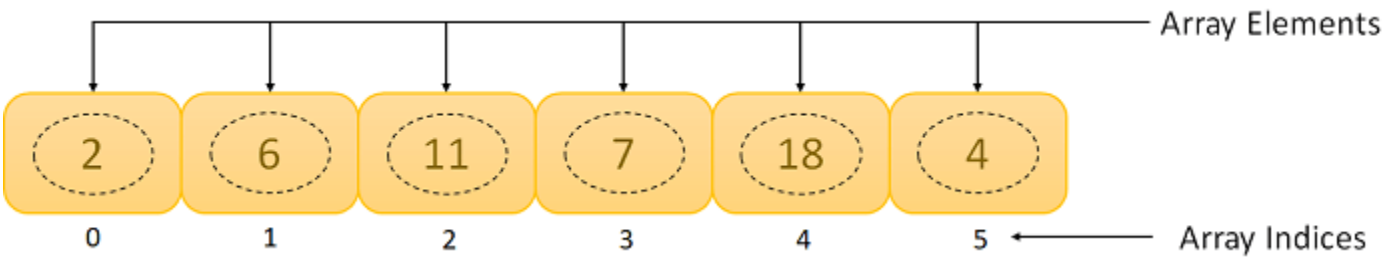


Figure. An Array

Representation of Arrays:

Declaring an array means specifying the following:

- *Data type* — the kind of values it can store, for example, int, char, float, double.
- *Name* — to identify the array.
- *Size* —the maximum number of values that the array can hold.

In C, arrays are declared using the following syntax:

```
data-type name[size];
```

For example,

```
int marks[10];
```

The above statement declares an array *marks* that contains 10 elements.

In C, the array index starts from zero. This means that the array marks will contain 10 elements in all. The first element will be stored in marks[0], second element in marks[1], so on and so forth. Therefore, the last element, that is the 10th element, will be stored in marks[9].

In the memory, the array will be stored in consecutive memory locations as shown in Fig. 1.1. The computer need not keep track of the address of every element of the array, but needs to keep track only of the address of the first element, called the **base address** of the array.

1st element	2nd element	3rd element	4th element	5th element	6th element	7th element	8th element	9th element	10th element
marks[0]	marks[1]	marks[2]	marks[3]	marks[4]	marks[5]	marks[6]	marks[7]	marks[8]	marks[9]

Figure 1.1 Memory representation of an array of 10 elements

The array name denotes

i) the name of the array
ii) the address of the array
iii) the address of the first byte of the array.

The subscript or index represents the offset from the beginning of the array to the element being referenced. Thus, with just the array name and the index, the computer can calculate the address of any element in the array, since an array stores all its data elements in consecutive memory locations. The address of the data elements can simply be calculated using the base address.

The compiler calculates the address of any element of **A** by the following formula:

Address(**A**[K]) = Base(**A**) + w * (K – LB)

Where Base(**A**) denotes the base address of the array **A**,

K is any subscript,

w is the number of words per memory cell for the array **A, and**

LB is the lower bound of the array**.**

The time to calculate Address(**A**[K]) is essentially the same for any value of K. Further, given any subscript K, one can locate and access the content of **A**[K] without scanning any other element of **A.**

Ex. 1.1 Given an array int marks[]={99,67,78,56,88,90,34,85}, calculate the address of marks[4] if the base address = 1000.

Solution: The size of an integer value in C requires 2 bytes and LB of a C array is 0.

Therefore, the address of marks[4] = 1000 + 2 * (4 – 0) = 1000 + 8 = 1008

Fig. 1.2 shows the storage representation of array marks in memory.

99	67	78	56	**88**	90	34	85
marks[0]	marks[1]	marks[2]	marks[3]	**marks[4]**	marks[5]	marks[6]	marks[7]
1000	1002	1004	1006	**1008**	1010	1012	1014

Fig. 1.2 Storage representation of array marks in memory

Arrays have the following **limitations**:

i) Arrays are of fixed size.
ii) Data elements are stored in contiguous memory locations which may not be always available.
iii) Insertion and deletion of elements can be problematic because of shifting of elements from their positions.

However, these limitations can be solved by using linked lists.

Arrays can be classified into different types:

a. **One-Dimensional Array:** An Array with only one row of data elements is known as a One-Dimensional Array. It is stored in ascending storage location.

b. **Two-Dimensional Array:** An Array consisting of multiple rows and columns of data elements is called a Two-Dimensional Array. It is also known as a Matrix.

c. **Multidimensional Array:** We can define Multidimensional Array as an Array of Arrays. Multidimensional Arrays are not bounded to two indices or two dimensions as they can include as many indices are per the need.

Some Applications of Array:

a. We can store a list of data elements belonging to the same data type.

b. Array acts as an auxiliary storage for other data structures.

c. The array also helps store data elements of a binary tree of the fixed count.

d. Array also acts as a storage of matrices.

STORING VALUES IN ARRAYS

After an array is declared, its elements must be initialized. Otherwise, they will contain "garbage".

An array can be initialized at either of the following stages:

- At compile time
- At run time

Compile Time Initialization

we can initialize the elements of arrays in the same way as the ordinary variables when they are declared. The general form of initialization of arrays is:

1) Full initailization

type array-name[size] = {list of values};

The values in the list are separated by commas. For example, the statement

int a[3] = { 1,2,3 }; will assign a[0]=1,a[1]=2,a[2]=3 with size 3.

2) Partial initailization

Compile time initialization may be partial. That is, the number of initializers may be less than declared size. In such cases. the remaining elements are inilialized to zero, if the array type is numeric and NULL if the type is char.

For example,

int number [5] = {10, 20}; will initialize the first two elements to 10 and 20 respectively, and the remaining elements to 0.

Similar the declaration. char city [5] = {‘B’}; will initialize the first element to 'B' and the remaining four to NULL.

3) **Without size**

The size may be omitted. In such cases, the compiler allocates enough space for all initialize elements. For example, the statement int a[ ] = {1,1,1,1}; will declare the **a** array to contain four elements with initial values 1. This approach works fine as long as we initialize every element in the array.

4) With character constants

Character arrays may be initialized in a similar manner. Thus, the statement

char name [ ] = { 'J' , 'o ‘ , ‘ h ‘, ‘ n ‘, ‘\0’}; declares the name to be an array of five characters, initialized with the string "John" ending with the null character.

5) With string

We can assign the string literal directly as under: char name [] = "John"; In this size is five. Here null character need not be explicitly specified.

If we have more initializers than the declared size, the compiler will produce an error. That is, the statement int number [3] = {10, 20, 30, 40}; will not work. It is illegal in C.

Run Time Initialization

An array can be explicitly initialized at run time. This approach is usually applied for initializing large arrays.

int a [3] ;

scanf("%d%d%d", &a[0],&a[1],&a[2]);

(or)

```
int i,a [3] ;
for(i=0;i<3;i++)
scanf("%d", &a[i]);
```

will initialize array elements with the values entered through the keyboard.

OPERATIONS ON ARRAYS

The following are the operations done on arrays

1) Traversing an array
2) Inserting an element in an array
3) Deleting an element in any array
4) Merging two arrays
5) Searching an element in an array
6) Sorting an array in ascending or descending order

1) Traversing an Array

Traversing an array means accessing each and every element of the array for a specific purpose.

1. Input and Output an array elements of size n

```
#include <stdio.h>
void main()
{
    int n,i,a[10];
    printf("enter size");
    scanf("%d",&n);
    printf("enter the elements");
    for(i=0;i<n;i++)
          scanf("%d",&a[i]);
    printf("The elements are");
    for(i=0;i<n;i++)
          printf("%d\t",a[i]);
}
```

2. Biggest element in an array

```
void main()
{
    int n,i,a[10],big;
    printf("enter size");
    scanf("%d",&n);
    printf("enter the elements");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
```

```
    big=a[0];
    for(i=0;i<n;i++)
    {
        if(a[i]>big)
        big=a[i];
    }
    printf("The biggest element is %d ",big);
}
```

3. Sum of array elements

```
#include <stdio.h>
void main()
{
    int n,i,a[10],sum=0;
    printf("enter size");
    scanf("%d",&n);
    printf("enter the elements");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    for(i=0;i<n;i++)
    {
        sum=sum+a[i];
    }
    printf("The sum is %d ",sum);
}
```

4. Sum of squares

Write a program using a single-subscripted variable to evaluate the following expression:

$$\text{Total} = \sum_{i=1}^{10} x_i^2$$

where the values of x_1 ,x_2, ... are read from the terminal.

```
#include <stdio.h>
void main()
{
    int n,i,a[10],sum=0;
    printf("enter size");
    scanf("%d",&n);
    printf("enter the elements");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
```

```
    for(i=0;i<n;i++)
    {
        sum=sum+a[i]*a[i];
    }
    printf("The sum is %d ",sum);
}
```

5. Add two array elements

```
#include <stdio.h>
void main()
{
    int n,i,a[10],b[10],c[10];
    printf("enter size");
    scanf("%d",&n);
    printf("enter the elements of a array");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    printf("enter the elements of b array");
    for(i=0;i<n;i++)
        scanf("%d",&b[i]);
    for(i=0;i<n;i++)
        c[i]=a[i]+b[i];
    printf("The resultant array is");
    for(i=0;i<n;i++)
        printf("%d",c[i]);
}
```

2) Inserting an Element in an Array

It means adding a new data element to an already existing array. Inserting an element at the end of array of **size n** is easy. If a is the array, a[n]=num and n=n+1.

Inserting in the middle of the array, say pos is also possible, for which we have to move all elements that have index greater than pos one position towards right and create space for the new element. Then we increment n by 1.

```
void main()
{
    int n,i,a[10],pos,num;
    printf("\n Enter size");
    scanf("%d",&n);
    printf("enter the elements");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
```

```
    printf("\n Enter num to insert");
    scanf("%d",&num);
    printf("\n Enter position to insert");
    scanf("%d",&pos);
    for(i=n-1;i>=pos;i--)
        a[i+1]=a[i];
    a[pos]=num;
    n++;
    printf("The resultant array is");
    for(i=0;i<n;i++)
        printf("\t %d",a[i]);
}
```

3) Deleting an Element in an Array

- It means deleting a data element from an existing array.
- Deleting an element at the end of array of **size n** is easy - simply decrement n as n=n-1.
- Deletion in the middle of the array is also possible, for which we have to move all elements that have index greater than pos one position towards left. Then we decrement n by 1.

```
#include <stdio.h>
void main()
{   int n,i,a[10],pos,num;
    printf("\n Enter size");
    scanf("%d",&n);
    printf("enter the elements");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    printf("\n Enter position to delete");
    scanf("%d",&pos);
    num = a[pos];
    printf("The element that is deleted = %d", num);
    for(i=pos;i<n-1;i++)
        a[i]=a[i+1];
    n--;
    printf("The resultant array is");
    for(i=0;i<n;i++)
        printf("\t %d",a[i]);
}
```

4) Merging Two Arrays

Merging two arrays in a third array means first copying the contents of first array into the third array and then copying the contents of the second array into the third array. Hence, the merged array contains contents of the first array followed by the contents of the second array.

```
void main()
{
    int a1[10],a2[10],a3[20],n1,n2,i,m,index=0;
    printf("\n Enter size of array1");
    scanf("%d",&n1);
    printf("\n Enter the elements of first array");
    for(i=0;i<n1;i++)
        scanf("%d",&a1[i]);
    printf("\n Enter size of array2");
    scanf("%d",&n2);
    printf("\n Enter the elements of second array");
    for(i=0;i<n2;i++)
        scanf("%d",&a1[i]);
    for(i=0;i<n1;i++)
    {
        a3[index]=a1[i];
        index++;
    }
    for(i=0;i<n2;i++)
    {
        a3[index]=a2[i];
        index++;
    }
    m=n1+n2;
    printf("The resultant array is");
    for(i=0;i<m;i++)
        printf("\t %d",a3[i]);
}
```

5) Searching for a value in an array

Searching means to find whether a particular value is present in the array or not.

Two methods:

1) Linear search
2) Binary Search

Linear Search

- Linear search is a very simply search algorithm where search is made over all items one by one.
- Every item is checked and if a match is found then that particular index is returned otherwise the search continues till the end of the array.

Algorithm Linear Search

```
Step 1: Read n and initialize found=0.
Step 2: Read array elements A.
Step 3: Read the key item to be searched.
Step 4: for i=0 till n-1
            if A[i] = key
          then
               print element found at index i;
               found=1;
                break;
          end if
        end for
Step 5: if found=0
        then
            print element not found
Step 6: stop
```

Example:

List of elements:

10	12	3	5	11	4	2	23	43	29

Key: 4

List of elements:

10	12	3	5	11	4	2	23	43	29 – Array elements
0	1	2	3	4	5	6	7	8	9 - index

Key: 4 found = 0

i = 0

10	12	3	5	11	4	2	23	43	29 – Array elements
0	1	2	3	4	5	6	7	8	9 - index

Key: 4 found = 0

i = 1

10	12	3	5	11	4	2	23	43	29 – Array elements
0	1	2	3	4	5	6	7	8	9 - index

Key: 4 found = 0

i = 2

10	12	3	5	11	4	2	23	43	29 – Array elements
0	1	2	3	4	5	6	7	8	9 - index

Key: 4 found = 0

i = 3

10	12	3	5	11	4	2	23	43	29 – Array elements
0	1	2	3	4	5	6	7	8	9 - index

Key: 4 found = 0

i = 4

10	12	3	5	11	4	2	23	43	29 – Array elements
0	1	2	3	4	5	6	7	8	9 - index

Key: 4 found = 0

i = 5

10	12	3	5	11	4	2	23	43	29 – Array elements
0	1	2	3	4	5	6	7	8	9 - index

Key: 4 found = 1

i = 5

10	12	3	5	11	4	2	23	43	29 – Array elements
0	1	2	3	4	5	6	7	8	9 - index
1	2	3	4	5	6	7	8	9	10 - position

Element found at position i+1

LINEAR SEARCH – PROGRAM

```
#include<stdio.h>
void main( )
```

```
{
    int n, a[20], found=0, i, key;
    printf("enter the number of elements n");
    scanf("%d",&n);
    printf("enter the array elements");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    printf("enter the key to be searched\n");
    scanf("%d",&key);

    for(i=0;i<n;i++)
    {
        if(a[i]==key)
        {
            printf("element found at position %d\n",i+1);
            found=1;
            break;
        }
    }
    if(found==0)
        printf("element not found");
}
```

Advantages

Linear search is simple and very easy to understand and implement

It does not require the data in the array to be stored in any particular order.

Disadvantages

It is inefficient

For example, if we have 1000 elements and the key element is found at last index then it looks 1000 elements in order to find the last element.

Binary Search

- Binary search looks for a particular item by comparing the middle most item of the sorted array. If a match occurs, then the index of item is returned.
- If the middle item is greater than the item, then the item is searched in the sub-array to the left of the middle item.
- Otherwise, the item is searched for in the sub-array to the right of the middle item. This process continues on the sub-array as well until the size of the sub-array reduces to zero.

Algorithm - Binary Search

Step 1: read n and set found=0

```
Step 2: read array elements A in ascending Order
Step 3: read the key item to be searched
Step 4: set low=0
set high=n-1
Step 5: while key not found and low<=high
do
    mid=(low+high)/2
    if A[mid] = key
        set found=1
    if a[mid] < key
        set low = mid +1
    if a[mid] > key
        set high = mid -1
end while
step 6: if found =1
          then
                    print element found at position mid+1
          else
                    print element not found
step 7: stop.
```

Working

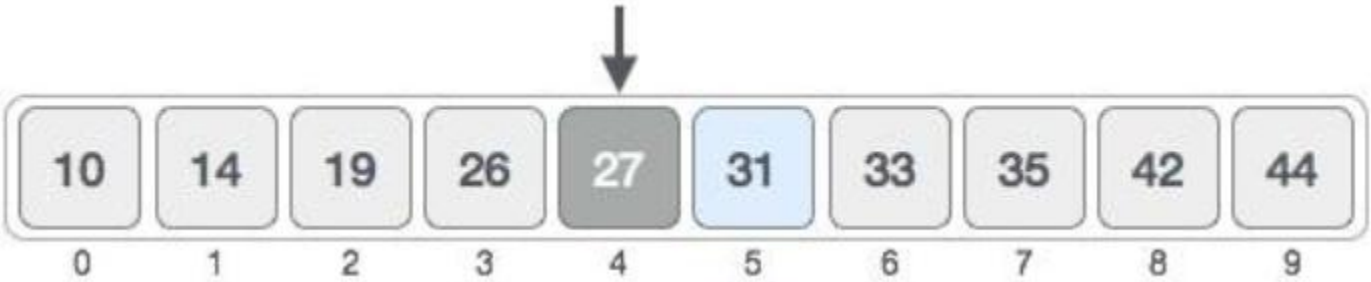
For a binary search to work, it is mandatory for the target array to be sorted.

The following is our sorted array and let us assume that we need to search the location of value 31 using binary search.

First, we shall determine half of the array by using this formula

mid=(low+high)/2

Here it is, (0 + 9) / 2 = 4 (integer value of 4.5). So, 4 is the mid of the array.



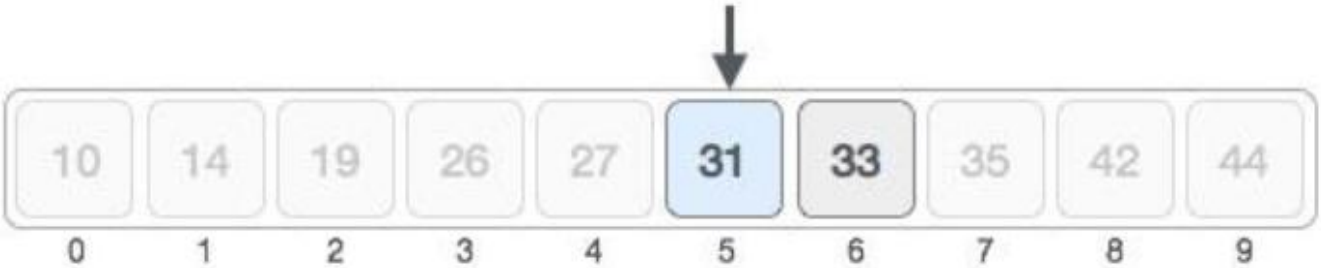
Now we compare the value stored at location 4, with the value being searched, i.e. 31.

We find that the value at location 4 is 27, which is not a match. As the value is greater than 27 and we have a sorted array, so we also know that the target value must be in the upper portion of the array.

We change our low to mid + 1 and find the new mid value again.
Our new mid is 7 now. We compare the value stored at location 7 with our target value 31.

The value stored at location 7 is not a match, rather it is more than what we are looking for. So, the value must be in the lower part from this location.

Hence, we calculate the mid again. This time it is 5.



We compare the value stored at location 5 with our target value. We find that it is a match.

We conclude that the target value 31 is stored at location 5.

```
#include<stdio.h>
void main()
{
  int n,a[20],found=0,i,key,low,high,mid;
  printf("enter the number of elements n");
  scanf("%d",&n);
```

```
  printf("enter the array elements in ascending order");
  for(i=0;i<n;i++)
    scanf("%d",&a[i]);
  printf("enter the key to be searched\n");
  scanf("%d",&key);
  low=0;
  high=n-1;
  while(low<=high)
  {
    mid=(low+high)/2;
    if(a[mid]==key)
    {
      printf("element found at position %d\n", mid+1);
      found=1;
      break;
    }
    else if(a[mid]<key)
      low=mid+1;
    else
      high=mid-1;
  }
  if(found==0)
    printf("\n element not found at position”);
}
```

Sorting

Arranging the elements of an array either in ascending or in descending order is called sorting an array.

Bubble sort

Bubble sort is a simple sorting algorithm. This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order.

Working

Take an unsorted array for an example.

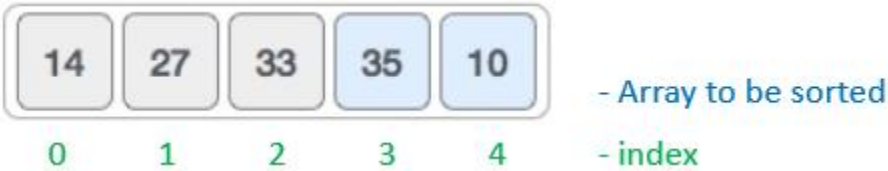
Bubble sort starts with very first two elements, comparing them to check which one is greater.

In this case, value 33 is greater than 14, so it is already in sorted locations. Next, we compare 33 with 27.
In this case, value 33 is greater than 14, so it is already in sorted locations.
Next, we compare 33 with 27.

We find that 27 is smaller than 33 and these two values must be swapped.
The new array:

Next we compare 33 and 35.
We find that both are in already sorted positions.

Then we move to the next two values, 35 and 10.



We know then that 10 is smaller 35. Hence they are not sorted.
We swap these values.

We find that we have reached the end of the array.

Step 1: read n

Step 2: read array elements A in unsorted order
Step 3: print unsorted array elements A
Step 4: for i=0 till n-1
do
for j=0 till n-i-1
do
    if A[j] > A[j+1]
        swap (A[j] , A[j+1]
    end if
    end for
end for
step 5: print the sorted array elements A
step 6: stop

Binary Search Program

```
#include<stdio.h>
int main()
{
  int n,a[20],i,j, temp;
  printf("enter the number of elements n");
  scanf("%d",&n);
  printf("enter the array elements in any order");
    for(i=0;i<n;i++)
      scanf("%d",&a[i]);
    printf("the unsorted elements are \n");
    for(i=0;i<n;i++)
      printf ("%d\t",a[i]);
    for(i=0;i<n-1;i++)
    {
      for(j=0;j<n-i-1;j++)
      {
        if(a[j]>a[j+1])
        {
          temp=a[j];
          a[j]=a[j+1];
          a[j+1]=temp;
        }
      }
    }
    printf("\n the sorted elements are\n");
    for(i=0;i<n;i++)
      printf ("%d\t",a[i]);
  }
```

```
}
```

Program 3 Bubble Sort

```
#include<stdio.h>
void bubblesort(int a[],int n);
void bubblesort(int a[],int n)
{
	for(i=0;i<n-1;i++)
	{
		for(j=0;j<n-i-1;j++)
		{
			if(a[j]>a[j+1])
			{
				temp=a[j];
				a[j]=a[j+1];
				a[j+1]=temp;
			}

		}
	}

}
int main()
{
	int n,a[20],i,j, temp;
	printf("enter the number of elements n");
	scanf("%d",&n);
	printf("enter the array elements in any order");
	for(i=0;i<n;i++)
	{
		scanf("%d",&a[i]);
	}
	printf("the unsorted elements are \n");
```

```
        for(i=0;i<n;i++)
        {
                printf ("%d\t",a[i]);
        }
        bubblesort(a,n);
        printf("\n the sorted elements are\n");
        for(i=0;i<n;i++)
        {
                printf ("%d\t",a[i]);
        }
        return 0;
}
```

BUBBLE SORT – Function Call -PROGRAM

```
#include<stdio.h>
void bubblesort(int a[],int n);
void bubblesort(int a[],int n)
{
     int i,j,temp;
     for(i=0;i<n-1;i++)
     {
        for(j=0;j<n-i-1;j++)
        {
          if(a[j]>a[j+1])
          {
              temp=a[j];
              a[j]=a[j+1];
              a[j+1]=temp;
           }
        } /* for */
      }   /* for */
     }     /* bubblesort */
     int main()
     {
          int n,a[20],i,j, temp;
          printf("enter the number of elements n");
          scanf("%d",&n);
          printf("enter the array elements in any order");
          for(i=0;i<n;i++)
               scanf("%d",&a[i]);
```

```
        printf("the unsorted elements are \n");
        for(i=0;i<n;i++)
        {
            printf("%d\t",a[i]);
        }
        bubblesort(a,n);
        printf("\n the sorted elements are\n");
        for(i=0;i<n;i++)
        {
            printf ("%d\t",a[i]);
        }
        return 0;
}
```

Selection Sort Algorithm

Selection sort *is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the list and moving it to the sorted portion of the list.*

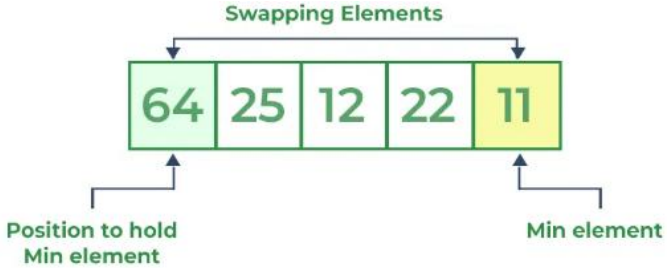
The algorithm repeatedly selects the smallest (or largest) element from the unsorted portion of the list and swaps it with the first element of the unsorted part. This process is repeated for the remaining unsorted portion until the entire list is sorted.

<u>How does Selection Sort Algorithm work?</u>

Lets consider the following array as an example: ***arr[] = {64, 25, 12, 22, 11}***

First pass:

- *For the first position in the sorted array, the whole array is traversed from index 0 to 4 sequentially. The first position where* ***64*** *is stored presently, after traversing whole array it is clear that* ***11*** *is the lowest value.*
- *Thus, replace 64 with 11. After one iteration* ***11****, which happens to be the least value in the array, tends to appear in the first position of the sorted list.*



Selection Sort Algorithm | Swapping 1st element with the minimum in array

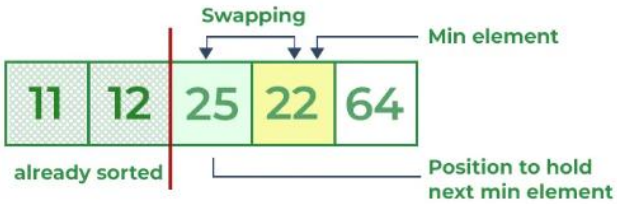
Second Pass:

- *For the second position, where 25 is present, again traverse the rest of the array in a sequential manner.*
- *After traversing, we found that* ***12*** *is the second lowest value in the array and it should appear at the second place in the array, thus swap these values.*

Selection Sort Algorithm | swapping i=1 with the next minimum element

Third Pass:

- *Now, for third place, where **25** is present again traverse the rest of the array and find the third least value present in the array.*
- *While traversing, **22** came out to be the third least value and it should appear at the third place in the array, thus swap **22** with element present at third position.*



Selection Sort Algorithm | swapping i=2 with the next minimum element

Fourth pass:

- *Similarly, for fourth position traverse the rest of the array and find the fourth least element in the array*
- *As **25** is the 4th lowest value hence, it will place at the fourth position.*

Selection Sort Algorithm | swapping i=3 with the next minimum element

Fifth Pass:

- *At last the largest value present in the array automatically get placed at the last position in the array*
- *The resulted array is the sorted array.*

Selection Sort Algorithm | Required sorted array

Program 4: Write a program in C to create a data structure which can handle varying number of entries and sort it.

```
#include <stdio.h>
void selectionsort(int a[],int n);
void selectionsort(int a[],int n)
{
```

```
        int i,j,pos,temp;
        for(i=0;i<n-1;i++)
        {
                pos=i;
                for(j=i+1;j<n;j++)
                {
                        if(a[j]<a[pos])
                                pos=j;
                }
                if(i!=pos)
                {
                        temp=a[i];
                        a[i]=a[pos];
                        a[pos]=temp;
                }
        }
}
int main()
{
        int a[100],n,i;
        printf("enter the number of elements\n");
        scanf("%d",&n);
        printf("enter the array elements\n");
        for(i=0;i<n;i++)
        scanf("%d",&a[i]);
        selectionsort(a,n);
        printf("after sorting\n");
        for(i=0;i<n;i++)
                printf("%d\t",a[i]);
        return 0;
}
```

Abstract Data Type – 1D Array

1. **Data**: Collection of elements of the same type.
2. **Size/Capacity**: Maximum number of elements the array can hold.
3. **Operations** (functions allowed on the array):
    1. Create(n) → Create an array of size n.
    2. Insert(index, element) → Insert an element at a specific position.
    3. Delete(index) → Delete element at a specific position.
    4. Update(index, element) → Modify element at a specific position.
    5. Get(index) → Retrieve element at a specific position.

6. Display() → Print all elements.
7. Search(element) → Find position(s) of a given element.
8. Sort() -> Arranges the elements of the array in ascending/descending order

Review of Pointers

A **pointer** is a variable that **stores** the **memory address** of another variable.

A **pointer variable points** to a **data type** (like int) of the same type and is created with the * operator.

The address of the variable you are working with is assigned to the pointer.

The two most important operators used with the pointer type are

- & - The "**address of**" operator - The unary operator & which gives the address of a variable
- *- The **indirection** or **dereference operator** * gives the content of the object pointed to by a pointer. The indirection operator * gets the value stored in the memory location whose address is stored in a pointer variable. The address of & operator returns the address of the memory location in which the variable is stored.

Ex- int i, *pi;
Means i is an integer variable and pi is a pointer to an integer.
pi= &i;
Then &i returns the address of i and assigns it as the value of pi.

The use of pointers in C can be divided into three steps:
1. **Pointer Declaration**
2. **Pointer Initialization**
3. **Pointer Dereferencing**

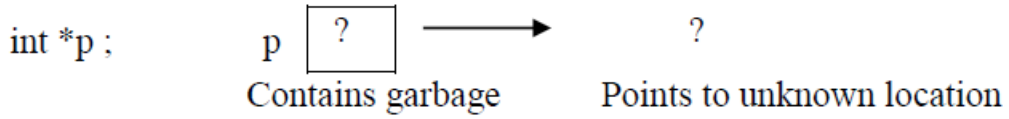
<u>Pointer Declaration</u>
In pointer declaration, we only declare the pointer but do not initialize it. To declare a pointer, we use the **(*) dereference operator** before its name.

The general syntax of declaring pointer variables:
data_type *ptr_name;

Here, data_type is the data type of the value that the pointer will point to.

Example:
int *pnum;
char *pch;
float *pfnum;

- In each of the above statements, a pointer variable is declared to point to a variable of the specified data type.
- The declarations cause the compiler to allocate memory locations for the pointer variables p.
- Since the memory locations have not been assigned any values i.e. initialized, these locations may contain some unknown values in them and therefore they point to unknown locations. Such pointers are called **wild pointers**.



Pointer Initialization

Pointer initialization is the process of assigning the address of a variable to the pointer variable.

We generally use the **(&: ampersand) address of operator** to get the memory address of a variable and then store it in the pointer variable.

int var = 10;
int * **ptr**;
ptr = **&**var;

We can also combine **the initialization with the declaration**. That is,

int *ptr = &x;

is allowed. **The only requirement here is that the variable x must be declared before the initialization takes place.**

```
#include<stdio.h>
int main()
{
  int num, *ptr;
  ptr = &num;
  printf("\n Enter the number : ");
  scanf("%d", &num);
  printf("\n The number that was entered is : %d", *ptr);
  printf("\n The address of number that was entered is : %d", ptr);
  return 0;
}
```

Output:

Enter the number : 10

The number that was entered is : 10

The address of number that was entered is : 6487572

What will be the value of *(&num)?

It is equivalent to simply writing num.

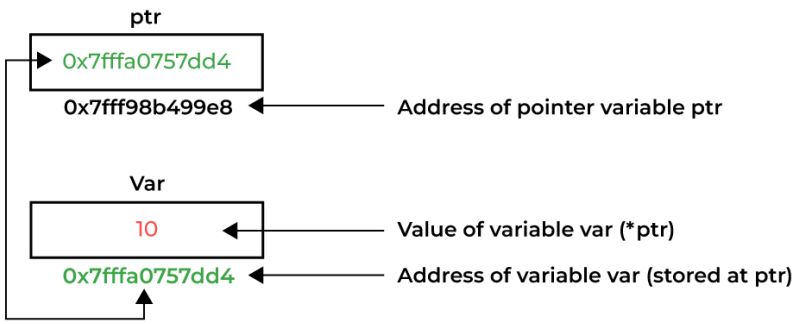
Example

```
int myAge = 43;     // An int variable
int* ptr = &myAge; // A pointer variable, with the name ptr, that stores the address of myAge

// Output the value of myAge (43)
printf("%d\n", myAge);
```

```
// Output the memory address of myAge (0x7ffe5367e044)
printf("%p\n", &myAge);

// Output the memory address of myAge with the pointer (0x7ffe5367e044)
printf("%p\n", ptr);
```

Pointer Dereferencing

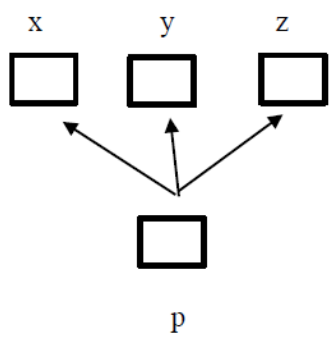
Dereferencing a pointer is the process of accessing the value stored in the memory address specified in the pointer. We use the same **(*) dereferencing operator** that we used in the pointer declaration.



POINTER FLEXIBILITY Pointers are flexible.

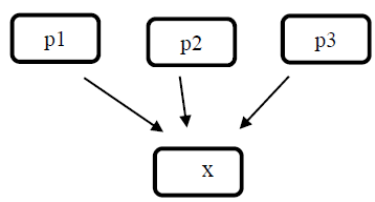
- The same pointer can be made to point to different data variables in different statements. Example;

```
int x, y, z, *p;
p = &x;
...
p = &y;
...
p = &z;
```



- Different pointers can be used to point to the same data variable. Example;

```
int x;
int *pl = &x;
int *p2 = &x;
int *p3 =&x;
```



With the exception of NULL and 0, no other constant value can be assigned to a pointer variable.
For example, the following is invalid:
int *p = 5360; / *absolute address */

Differences between normal variable and a pointer variable

Normal Variable	Pointer Variable
Stores values	Stores addresses
Not dereferenced to print the value through variable.	Dereferenced with the indirection operator, *, to print the value (content) through pointer.
Dereferenced with the address of operator, &, to print the address through variable.	Not dereferenced to print the address through pointer,

Program to add 2 numbers using pointers

```
void main()
{
   int a, b, c, *p, *q;
   printf("enter 2 numbers");
   scanf("%d%d",&a,&b);
   p=&a;
   q=&b;
   c=*p + *q;
   printf("sum=%d", c);
}
```

TYPES OF POINTERS

Null Pointers

- A null pointer is a special pointer value and does not point to any value.
  A null pointer points to no object or function.
  This means that a null pointer does not point to any valid memory address.
- To declare a null pointer, we may use the predefined constant NULL
  Example: int *ptr = NULL;
- We can always check whether a given pointer variable stores the address of some variable or contains NULL by writing,

```
if (ptr == NULL)
{
   Statement block;
}
```

- The NULL pointer is represented by the integer 0. The C macro NULL is defined to be this constant.
- We may also initialize a pointer as a null pointer by using the constant 0

```
int *ptr;
ptr = 0;     /* This is a valid statement in C. */
```

- To test for the null pointer in C, we can say :

```
if (ptr == NULL)

if (!ptr)
```

Generic Pointers

- A generic pointer is a pointer variable that has void as its data type.

- The void pointer, or the generic pointer, is a special type of pointer that can point to variables of any data type.
- It is declared like a normal pointer variable but using the void keyword as the pointer's data type.
  For example: void *ptr;
- In C, since we cannot have a variable of type void, the void pointer will therefore not point to any data and, thus, cannot be dereferenced.
- We need to cast a void pointer to another kind of pointer before using it.
- Generic pointers are often used when you want a pointer to point to data of different types at different times.

```
#include<stdio.h>
int main()
{
  int x=10;
  char ch = 'A';
  void *gp;
  gp = &x;
  printf("\n Generic pointer points to the integer value = %d", *(int*)gp);
  gp = &ch;
  printf("\n Generic pointer now points to the character= %c", *(char*)gp);
  return 0;
}
```

Output

Generic pointer points to the integer value = 10

Generic pointer now points to the character = A

Double Pointers (Chain of Pointers or Pointers to Pointers)

Double pointers is used to make a pointer to point to another pointer, thus creating a chain of pointers.

```
void main()
{
  int x=10;
  int *p1,**p2;
  p1=&x;
  p2=&p1;
  printf("%d",**p2);
}
```

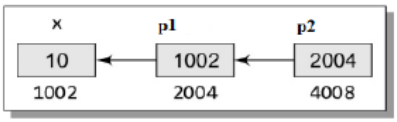


Figure Pointer to pointer

Output:

10

Here p2 contains address of p1. This is also known as multiple indirections.

int **p2; tells the compiler that p2 is a pointer to a pointer of int type.

POINTER ARITHMETIC (ADDRESS ARITHMETIC)

POINTERS INCREMENT AND SCALE FACTOR

Let us consider starting address as 8000. Then the operation p++ is done to point to next subsequent element:

1. int
   p++ (i.e) p=p+1 address is 8002.
2. char
   p++ (i.e) p=p+1 address is 8001.
3. float
   p++ (i.e) p=p+1 address is 8004.
4. double
   p++ (i.e) p=p+1 address is 8008.

Let us consider starting address as 8008.
Then the operation p-- is done to point to previous element:

1. int
   p-- (i.e) p=p-1 address is 8006.
2. char
   p-- (i.e) p=p-1 address is 8007.
3. float
   p-- (i.e) p=p-1 address is 8004.
4. double
   p-- (i.e) p=p-1 address is 8000.

Let us consider starting address as 8000.
Then the operation p+5 is done to point to the fifth element from that element (i.e) element at index 5:

1. int
   p=p+5 (i.e) p=p+5 (starting address+index*sizeof(int))=8000+5*2
   address is 8010.
2. char
   p=p+5 (i.e) p=p+5 (starting address+index*sizeof(char))=8000+5*1
   address is 8005.
3. float
   p=p+5 (i.e) p=p+5 (starting address+index*sizeof(float))=8000+5*4
   address is 8020.
4. double
   p=p+5 (i.e) p=p+5 (starting address+index*sizeof(double))=8000+5*8
   address is 8040.

Let us consider starting address as 8040.
Then the operation p-5 is done to point to the five elements before that element:

1. int
   p=p-5 (i.e) p=p-5 (starting address-index*sizeof(int))=8040-5*2
   address is 8030.
2. char
   p=p-5 (i.e) p=p-5 (starting address-index*sizeof(char))=8040-5*1
   address is 8035.
3. float
   p=p-5 (i.e) p=p-5 (starting address-index*sizeof(float))=8040-5*4

address is 8020.

4. double

   p=p-5 (i.e) p=p-5 (starting address-index*sizeof(double))=8040-5*8

   address is 8000.

When we increment a pointer, its value is increased by the length of the data type that its points. This length is called **scale factor**.

- Scale factor for **int** in a 16-bit machine is **2 bytes** (i.e) the size of the datatype.
- Scale factor for **char** in a 16-bit machine is **1 byte** (i.e) the size of the datatype.
- Scale factor for **float** in a 16-bit machine is **4 bytes** (i.e) the size of the datatype.
- Scale factor for **double** in a 16-bit machine is **8 bytes** (i.e) the size of the datatype.

Pointer Operations

1. Arithmetic operations.
2. A variable of type pointer may be assigned a pointer.
3. Arithmetic operations such as addition, subtraction, since a pointer is just a nonnegative integer number.
4. Relational operations – for example: we can determine if one pointer is greater than, less than, or equal to another pointer.
5. Pointers can be explicitly converted to integers.

Note: The size of a pointer can be different on different computers.

This artificial sequence shows how to declare a pointer and how to use & and *:

```
int x = 1, y = 2, z[10];
int *ip;        /* ip is a pointer to int */
ip = x;                 /* ip now points to x */
y = *ip;        /* y is now 1 */
*ip = 0;        /* x is now 0 */
ip = &z[0];     /* ip now points to z[0] */
```

If ip points to the integer x, then *ip can occur in any context where x could, so

```
*ip = *ip + 10;
```

increments *ip by 10.

The unary operators * and & bind more tightly than arithmetic operators, so the assignment

```
y = *ip + 1
```

takes whatever ip points at, adds 1, and assigns the result to y, while

```
*ip += 1
```

increments what ip points to, as do

```
++*ip
```

and

```
(*ip)++
```

The parentheses are necessary in this last example; without them, the expression would increment ip instead of what it points to, because unary operators like * and ++ associate right to left.

Finally, since pointers are variables, they can be used without dereferencing. For example, if iq is another pointer to int,

iq = ip

copies the contents of ip into iq, thus making iq point to whatever ip pointed to.

Drawbacks of Pointers

Although pointers are very useful in C, they are not free from limitations. If used incorrectly, pointers can lead to bugs that are difficult to unearth. For example, if you use a pointer to read a memory location but that pointer is pointing to an incorrect location, then you may end up reading a wrong value. An erroneous input always leads to an erroneous output. Thus however efficient your program code may be, the output will always be disastrous. Same is the case when writing a value to a particular memory location.

Some common errors when using pointers are:

1) **Assigning values to uninitialized pointers**

```
int x, *px;
x=10; *px = 20;
```

Error: Un-initialized pointer. px is pointing to an unknown memory location. Hence it will overwrite that location's contents and store 20 in it.

2) **Assigning a value to a pointer**

```
int x, *px;
x=10; px = x;
```

Error: It should be px = &x;

3) **Not dereferencing to print the value**

```
int x=10, y=20, *px, *py;
px = &x, py = &y;
if(px<py)
printf("\n x is less than y");
else
printf("\n y is less than x");
```

Error: It should be if(*px< *py)

4) **Assigning the address of uninitialized variable.**

```
int m, *p;
p=&m;
p=&m;
printf("%d",p);
```

5) Comparing pointers that point to different objects

```
char name1[20],name2[20];
char *p1=name1;
char  *p2=name2;
if(p1>p2)
```

Dynamic Memory Allocation

Sometimes you may have to allocate memory at execution. This feature is known as **dynamic memory allocation.**

Whenever we want to reserve memory locations for an object, it can be done in two ways:

1) Static memory allocation
2) Dynamic memory allocation

Static memory allocation requires that the declaration and definition of memory be fully specified in the source program. The number of bytes reserved cannot be changed during run time. Static allocation works well as long as you know exactly what your data requirements are.

Dynamic memory allocation uses predefined functions to allocate and release memory for data while the program is running. This effectively postpones the data definition at runtime. To use dynamic memory allocation, the programmer must use either standard data types or already must have declared any derived types.

Differences between static memory allocation and dynamic memory allocation

Static Memory Allocation	Dynamic memory allocation
Memory is allocated at compile time.	Memory is allocated at run time.
Memory cannot be increased or decreased while executing program.	Memory can be increased or decreased while executing program.
Requires that the declaration and definition of memory be fully specified in the source program.	Uses predefined functions to allocate and release memory for data while the program is running.
Works well as long as you know exactly what your data requirements are.	Works well when you do not know exactly what your data requirements are.
Used in arrays.	Used in linked lists.

- C provides a mechanism called a **heap** for allocating storage at runtime.
- Four memory management functions are used with dynamic memory – malloc, calloc, realloc and free.
- Three of these i.e. malloc, calloc and realloc are used for memory allocation and free is used to return memory when it is no longer needed.
- All the memory management functions are found in the standard library header file (<stdlib.h>).

1) Memory Allocation (malloc):

- The `malloc` **prototype**:

```
void *malloc(size_t size);
```

- The malloc function allocates a block of memory that contains the number of bytes specified in its parameter.
- It returns a void pointer to the first byte of allocated memory.
- So, the allocated memory should be type-casted to the kind of memory we want.
- If it is not successful, malloc returns a NULL pointer.
- The allocated memory is not initialized. Hence, you should assume that it will contain garbage and initialize it as required by your program.
- An attempt to allocate memory from the heap when memory is insufficient is known as **overflow.** It is upto the programmer to check for memory overflow.
- For example, if we want to allocate an integer:

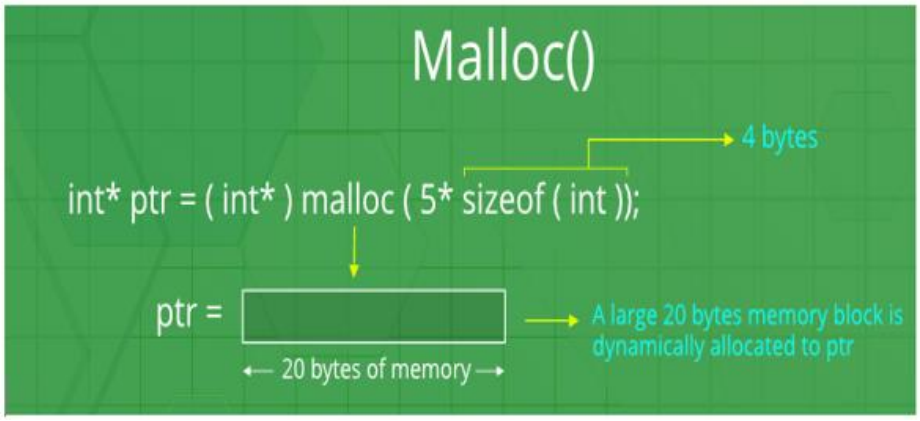
```
pInt = (int *) malloc(sizeof(int));
```

- Example: To check if memory has been allocated,

```
if (!(ptr = (int *) malloc(sizeof(int))))
        exit(100);
```

- If you call malloc with a zero size, the results are unpredictable. It may return a NULL pointer or it may return some other implementation-dependent value. **You should never call malloc with a zero size.**

int *ptr = (int*) malloc(5*sizeof(int));



Consider the following program that allocates and deallocates memory.

```
int i, *pi;
float f, *pf;
pi = (int *) malloc(sizeof(int));
pf = (float *) malloc(sizeof(float));
*pi = 1024;
*pf = 3.14;
printf("An integer = %d, a float = %f\n", *pi, *pf);
free(pi);
free(pf);
```

Since there is a possibility that a call to malloc may fail for lack of sufficient memory, a more robust version of the above program may be written by replacing the lines of code that invoke malloc by the code:

```
if ((pi = (int *) malloc(sizeif(int))) == NULL)
{
     printf("Insufficient memory");
     exit(EXIT_FAILURE);
}
if ((pi = (int *) malloc(sizeif(int))) == NULL ||
     pf = (float *) malloc(sizeof(float))) == NULL)
{
     fprintf(stderr, "Insufficient memory");
     exit(EXIT_FAILURE);
}
```

Since `malloc` may be invoked from several places in your program, define a macro that invokes `malloc` and exits when `malloc` fails.

The following is one such definition:

```
#define MALLOC(p,s) \
     if (!((p) = malloc(s))) { \
          fprintf(stderr, "Insufficient memory"); \
          exit(EXIT_FAILURE); \
```

```
}
```

This can be called in the above program fragment as follows:

MALLOC(pi, sizeof(int));

MALLOC(pf, sizeof(float));

2) Contiguous Memory Allocation (CALLOC):

- This is used to allocate memory for arrays.
- It differs from malloc in three ways:
  i. It allocates a contiguous block of memory large enough to contain an array of elements of a specified size.
  ii. `calloc` returns a pointer to the first element of the allocated array. `malloc` returns a void pointer to the first byte of the allocated memory.
  iii. It clears memory i.e. the memory it allocates is set to 0.
- Its prototype:

```
void *calloc(size_t element_count, size_t element_size);
```

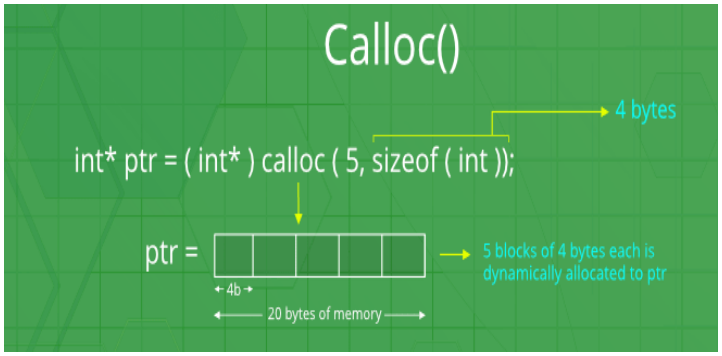
- A sample `calloc` call:

```
if(!(ptr = (int *) calloc(200, sizeof(int))))
   /* No memory available */
   exit(100);
/* Memory available */
```

- Another example:

```
int *x;
x = calloc(n, sizeof(int));
```

```
int *ptr = (int*) calloc(5,sizeof(int));
```

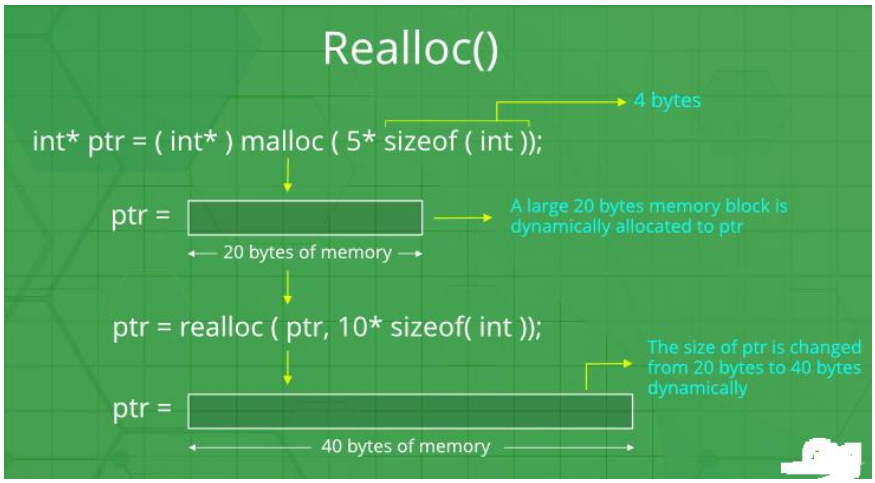


3) Reallocation of memory (REALLOC)

- When given a pointer to a previously allocated block of memory, `realloc` changes the size of the block by deleting or extending the memory at the end of the block.
- If the memory cannot be extended because of other allocations, `realloc` allocates a completely new block, copies the existing memory allocation to the new allocation and deletes the old allocation.
- The programmer must ensure that any pointers to the data are correctly changed.
- The `realloc` function can be highly inefficient and so should be used carefully.
- Its prototype:

```
void *realloc(void *ptr, size_t newSize);
```

```
ptr=realloc(ptr, 10*sizeof(int));
```



4) Free:

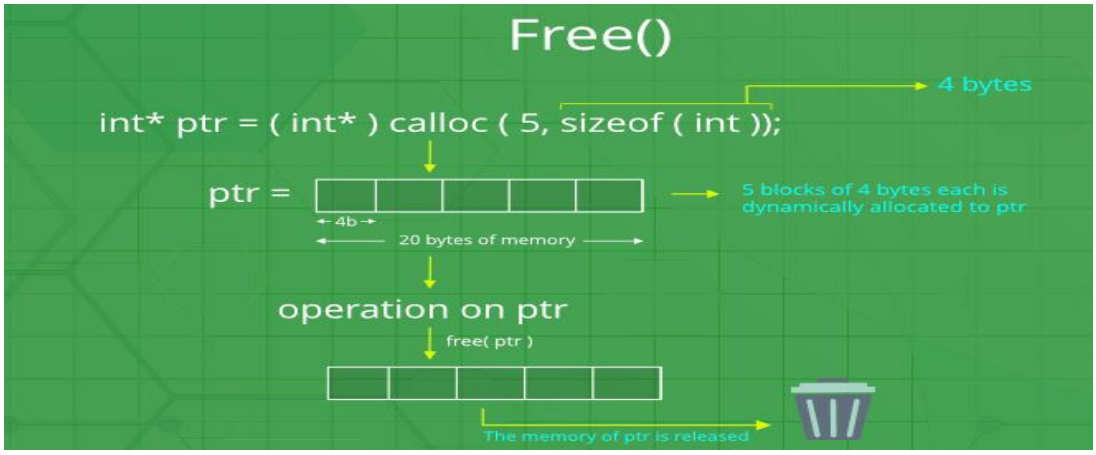
When

In some versions of C, free expects an argument that is a char *, while ANSI C expects void *. However, the casting of the argument is generally omitted in the call to free.

Its prototype:

void *free(void *ptr);

free(ptr) ;

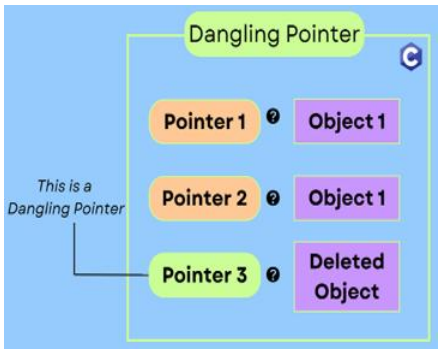


- The concept of **dynamic memory allocation in C language** enables the C programmer to allocate memory at runtime**.**
- To allocate some memory at run-time, C provides a mechanism called a **heap.**
- Dynamic memory allocation in C language is possible by 4 functions of stdlib.h header file.

malloc()	Allocates single block of requested memory
calloc()	Allocates multiple blocks of requested memory
realloc()	Reallocates the memory occupied by malloc() or calloc() functions
free()	Frees the dynamically allocated memory

Dangling Pointers

A **dangling pointer** in C is a pointer that points to a memory location that has been deallocated or is no longer valid. Dangling pointers can cause various problems in a program, including segmentation faults, memory leaks, and unpredictable behavior.



Example program – Allocation and deallocation of memory – Dangling Reference

```
int i, *pi;
float f, *pf;
pi = (int *) malloc(sizeof(int));
pf = (float *) malloc(sizeof(float));
*pi = 1024;
*pf = 3.14;
pf = (float *) malloc(sizeof(float));
*pf = 6.25;
printf("An integer = %d, a float = %f\n", *pi, *pf);
free(pi);
free(pf);
```

Due to the second malloc to pf, the pointer to the storage used to hold the value 3.14 has disappeared; there is no way to retrieve this storage. This is an example of *'dangling reference'.* Hence, it is a good practice to always return storage after we no longer need it.

POINTERS AND ARRAYS

int a[5]={10,20,30,40,50};

Suppose the base address is 8000, each integer requires two bytes.

Here a refers to starting address.
Also, &a[0] refers to the starting address.

Initialization:

int a[5];
int *p;
p=a;

(or)

int a[5];
int *p;
p=&a[0];

p=&a[0]	8000
p+1=&a[1]	8002
p+2=&a[2]	8004
p+3=&a[3]	8006
p+4=&a[4]	8008

Now, we can access the next element in the array by using p++.

```
printf("elements are: ");
for(i=0;i<n;i++)
{
        printf("%d\t",*p);
        p++;
}
```

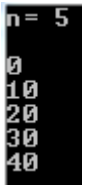
DYNAMICALLY ALLOCATED ARRAYS

ONE_DIMENSIONAL ARRAYS

During programming, we often find situations where we cannot reliably determine how large an array to use. A good solution is to defer this decision to run time and allocate the array when we have a good estimate of the required array size. The following is an example where the array list is dynamically allocated.

```
#include <stdio.h>
#include <stdlib.h>
main()
{
       int n, *list,i;
       printf("n= "); scanf("%d", &n);
       list = (int *) malloc(n* sizeof(int));
       for (i=0; i<n; i++)
              list[i] = 10*i;
       for(i=0; i<n; i++)
              printf("\n%d", list[i]);
       free(list);
getch();
}
```

Output:



Example 2

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
       int *ptr, i , n1, n2;
       printf("Enter size: ");
       scanf("%d", &n1);
       ptr = (int*) malloc(n1 * sizeof(int));
       printf("Enter Elements:\n");
       for(i = 0; i < n1; ++i)
           scanf("%d",ptr + i);
```

```
        printf("\nEnter the new size: ");
        scanf("%d", &n2);
        ptr = realloc(ptr, n2 * sizeof(int));
        printf("Enter Elements for new size:\n");
        for(i = 0; i < n2; ++i)
            scanf("%d",ptr + i);
        printf("Elements are:\n");
        for(i = 0; i < n2; ++i)
            printf("%d\t",*(ptr + i));
        free(ptr);
        return 0;
    }
```

Example program – Allocation and deallocation of memory

TWO-DIMENSIONAL ARRAYS

To represent multi-dimensional arrays, C uses array of arrays representation. Hence, a two-dimensional array is represented as a one-dimensional array in which each element is itself a one-dimensional array. For example, to represent the two-dimensional array

```
int nums[5][4];
```

we actually create a one dimensional array `nums` whose length is 5; each element of `nums` is a one-dimensional array whose length is 4. Fig. 1.14 shows the memory structure. Six separate memory blocks are used; one block is large enough for five pointers and each of the remaining blocks is large enough for four `ints`.

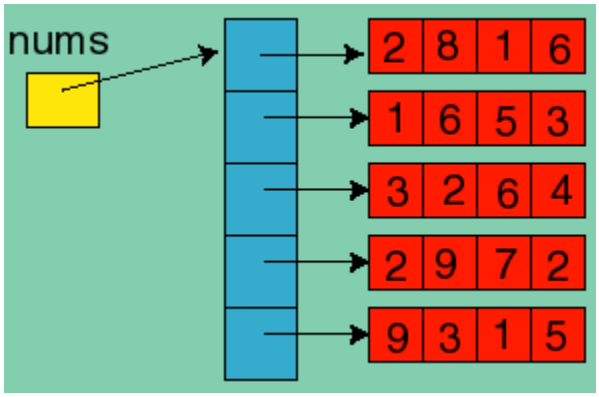


Fig. 1.14: Array-of-arrays representation

C finds the element x[i][j] by first accessing the pointer in x[i]. This pointer gives us the address in memory of the zeroth element of row i of the array. Then by adding j*sizeof(int) to this pointer, the address of the [j]th element of row i i.e. element x[i][j] is determined.

The following is an example program where the two-dimensional array arr is dynamically allocated.

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    int r = 3, c = 4;
    int *arr = (int *)malloc(r * c * sizeof(int));

    int i, j, count = 0;
    for (i = 0; i <  r; i++)
      for (j = 0; j < c; j++)
         *(arr + i*c + j) = ++count;

    for (i = 0; i <  r; i++)
      for (j = 0; j < c; j++)
         printf("%d ", *(arr + i*c + j));

   /* Code for further processing and free the
      dynamically allocated memory */

   return 0;
}
```

Output:

```
1 2 3 4 5 6 7 8 9 10 11 12
```

```
#include <stdio.h>
#include<stdlib.h>
int main()
{
    int r = 3, c = 4, i, j, count;
    int **arr;
    arr=(int **)malloc(r*sizeof(int **));
    for (i = 0; i < r; i++)
        arr[i]= (int*)malloc(c * sizeof(int *));
    count = 0;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            arr[i][j] = ++count;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            printf("%d\t", arr[i][j]);
    for (i = 0; i < r; i++)
        free(arr[i]);
    return 0;
}
```

Output:

Elements of User-Defined Functions

In order to make use of a user-defined function, we need to establish three elements that are related to functions.

- Function definition
- Function call
- Function declaration / function prototype

The ***function definition*** is written to implement the requirements of the function.

In order to use this function we need to invoke it at the required place in the program. This is known as ***function call***. The program (or a function that calls the function is referred to as the calling program or calling function.)

The calling program should declare any function (like declaration of a variable) that is used to be used later in the program. This is known as the ***function declaration or function prototype***.

Add two numbers

```
#include<stdio.h>

int add(int a,int b);

void main()
{
        int a,b,c;
        printf("Enter the value of a and b:");
        scanf("%d%d",&a,&b);
        c=add(a,b);
        printf("\n Addition of %d and %d is %d",a,b,c);
}

int add(int a, int b)
{
        int c;
        c=a+b;
        return c;
}
```

<u>Output</u>

Enter the values of a and b:4 5

Addition of 4 and 5 is 9

Parameter Passing techniques

The technique used to pass data from one function to another is known as parameter passing. Parameter passing can be done in two ways:

* Pass by value (also known as call by value).
* Pass by Pointers/Reference (also known as call by pointers).

Pass by value

In *pass by value,* values of actual parameters are copied to the variables in the parameter list of the called function. **Any changes in the formal arguments will not be reflected in the actual arguments.**

Example : To swap two numbers

```
void swap(int a,int b);
void main()
{
        int a,b;
        printf("enter two numbers");
        scanf("%d%d",&a, &b);
        printf("before swapping");
        printf("a=%d,b=%d",a,b);
        swap(a,b);
        printf("after swapping");
        printf("a=%d,b=%d",a,b);
}
void swap(int p,int q)
{
        int t;
        t=p;
        p=q;
        q=t;
        printf("in function");
        printf("a=%d,b=%d",a,b);
}
```

Output:

```
enter two numbers 5 4
before swapping
a=5,b=4
in function
a=4,b=5
after swapping
a=5,b=4
```

Advantages

The passing arguments can be variables, constants or expression

Disadvantages

Additional storage space, Lot of time to copy

Performance penalty if function called many times

Pass by pointers

In *pass by pointers* (also known as pass by address), the memory addresses of the variables is passed rather than the copies of values being sent to the called function.

Any changes in the formal arguments will be reflected in the actual arguments.

Example : To swap two numbers

```
void swap(int *p,int *q);
void main()
{
        int a,b;
        printf("enter two numbers");
        scanf("%d%d",&a,&b);
        printf("before swapping");
        printf("a=%d,b=%d",a,b);
        swap(&a,&b);
        printf("after swapping");
        printf("a=%d,b=%d",a,b);
}
void swap(int *p,int *q)
{
        int t;
        t=*p;
        *p=*q;
        *q=t;
}
```

Output:

enter two numbers 5 4
before swapping
a=5,b=4
after swapping
a=4,b=5

Advantages

Time and space efficiency

Change is reflected in the calling function

Used to return multiple values (pass those as arguments by reference)

Disadvantages

Difficult to tell whether the argument is meant for input, output or both.

Differences between Call by value and Call by reference

Call by value	Call by reference
Copy of values are passed from calling to called function.	Addresses are passed from calling to called function.
Any changes in the formal arguments will not be reflected in the actual arguments.	Any changes in formal arguments will be reflected in the actual arguments.
Execution of the program is slow.	Execution of the program Is fast.

Pointers and Function Arguments

Since C passes arguments to functions by value, there is no direct way for the called function to alter a variable in the calling function. For instance, a sorting routine might exchange two out−of−order arguments with a function called swap. It is not enough to write

swap(a, b);

where the swap function is defined as

```
void swap(int x, int y) /* WRONG */
{
   int temp;
   temp = x;
   x = y;
   y = temp;
}
```

Because of call by value, swap can't affect the arguments a and b in the routine that called it. The function above swaps *copies* of a and b.

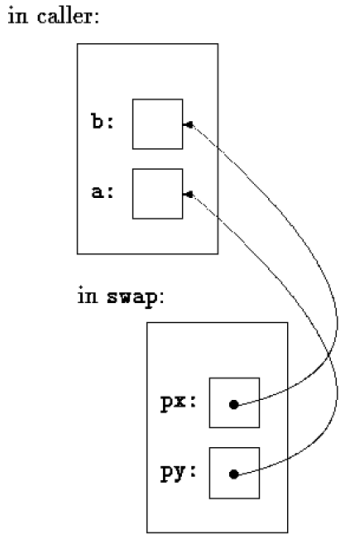
The way to obtain the desired effect is for the calling program to pass *pointers* to the values to be changed:

swap(&a, &b);

Since the operator & produces the address of a variable, &a is a pointer to a. In swap itself, the parameters are declared as pointers, and the operands are accessed indirectly through them.

```
void swap(int *px, int *py) /* interchange *px and *py */
{
   int temp;
   temp = *px;
   *px = *py;
   *py = temp;
}
```

Pictorially:



CAMBRIDGE
INSTITUTE OF TECHNOLOGY

Data Structures and Applications

Module 3(Linked Lists)

Department of Computer Science & Engineering

www.cambridge.edu.in

Module 3 – Linked Lists

Singly Linked list & Chains- Representation of Chains in C, Operations for Chains: Traversing, Searching, Insertion, and Deletion

Doubly Linked lists – Operations: Traversing, Searching, Insertion, and Deletion

Circular and header linked lists- Operations: Traversing, Searching, Insertion, and Deletion

Case Study- Polynomial using header linked list.

Text book:1

Linked Lists over Arrays

Why use linked list over array?

Array contains following limitations:

1. The size of array must be known in advance before using it in the program.
2. All the elements in the array need to be contiguously stored in the memory.
3. Inserting any element in the array needs shifting the elements to the right. Also, delete operations is tedious.
4. Expanding or shrinking the size of the array is difficult.

Linked list is the data structure which can overcome all the limitations of an array.

1. Using linked list is useful because, it allocates the memory dynamically.
2. All the nodes of linked list are non-contiguously stored in the memory and linked together with the help of pointers.
3. Sizing is no longer a problem since we do not need to define its size at the time of declaration.
4. List grows as per the program's demand and limited to the available memory space.

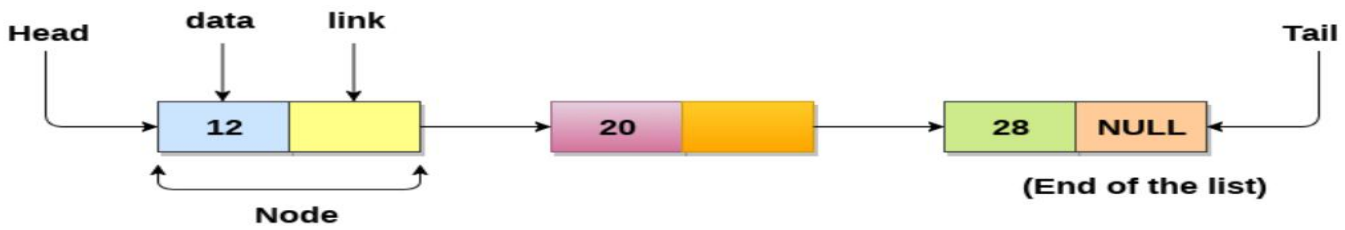
Note: No particular data structure is the best. The choice of the data structure depends on the kind of application that needs to be implemented. While for some applications linked lists may be useful, for others, arrays may be useful.

·**Use Arrays** when we need quick access to elements and when the data size is static.

·**Use Linked Lists** when the data size is dynamic, frequent insertions and deletions are needed, or when implementing certain complex data structures.

Definition : Linked List

Linked list is a linear data structure that consists of a sequence of elements where each element (usually called a node) comprises of two items - the data and a reference (link) to the next node. The last node has a reference to null.
The linked list contains a list variable called START or FIRST or HEAD, which contains the address of the first node in the list. We need only this address in START or HEAD to trace through the list.



The entry point into a linked list is called the **head (start or first)** of the list.

It should be noted that head is not a separate node, but the reference to the first node.

If the list is empty then the start is a null reference. The list with no nodes –empty list or null list.

Types of Linked List

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List
4. Header Linked List

Definition : Linked List

Representation of linked lists in Memory

We need the following capabilities to make linked representations possible:

- A mechanism for defining a node's structure, that is, the fields it contains. We use self-referential structure for this.

```
struct node
{
   int data;
   struct node *next;
};
struct node *head;
```

- A way to create new nodes when we need them. We use malloc() function for this.
- A way to remove nodes that we no longer need. We use free() function for this.

	DATA	LINK
1	HAT	15
2		
3	CAT	4
4	EAT	9
5		
6		
7	WAT	0
8	BAT	3
9	FAT	1
10		
11	VAT	7
	.	.
	.	.
	.	.

Figure 4.1 Non-Sequential List Representation

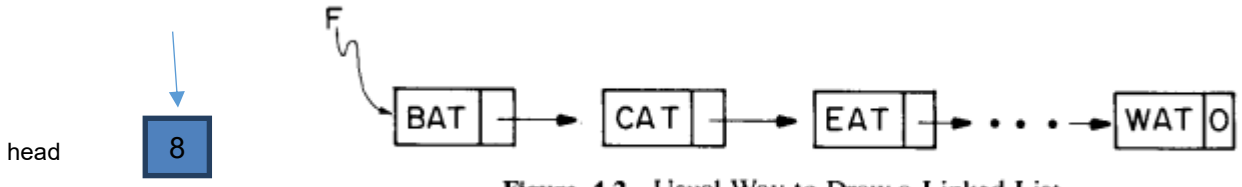


Figure 4.2 Usual Way to Draw a Linked List

Singly Linked List (Chains)

Singly linked list or One way chain (Representation of Chains in C)

Singly linked list can be defined as the collection of ordered set of elements. The number of elements may vary according to need of the program. A node in the singly linked list consist of two parts: data part and link part. Data part of the node stores actual information that is to be represented by the node while the link part of the node stores the address of its immediate successor.

One way chain or singly linked list can be traversed only in one direction. In other words, we can say that each node contains only next pointer, therefore we cannot traverse the list in the reverse direction.

Consider an example where the marks obtained by the student in three subjects are stored in a linked list as shown in the figure.

In the above figure, the arrow represents the links. The data part of every node contains the marks obtained by the student in the different subject. The last node in the list is identified by the null pointer which is present in the address part of the last node.

Singly Linked List (Chains) : Operations

Operations on Singly Linked List

There are various operations which can be performed on singly linked list.

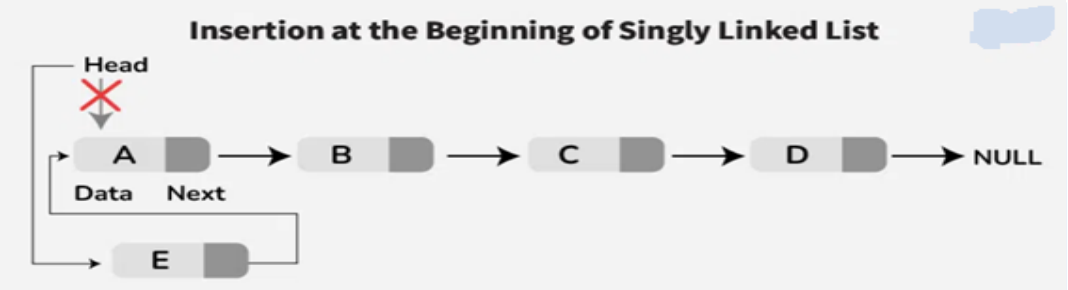
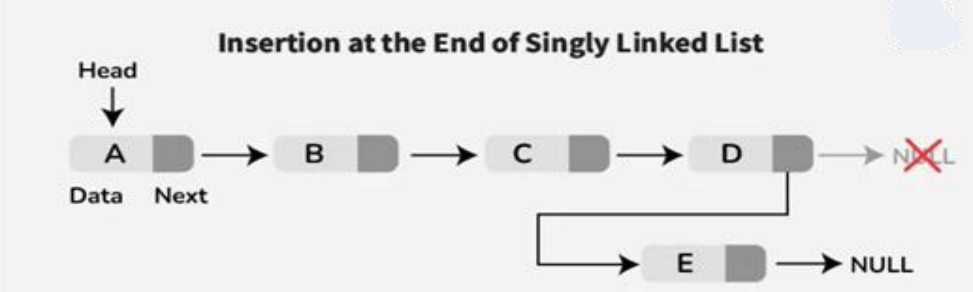
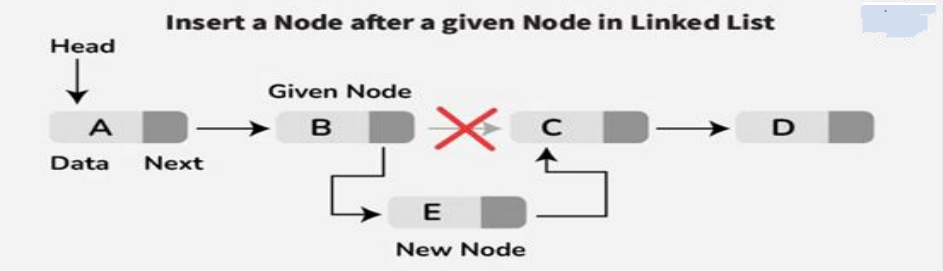
Operations :Traversing, Searching, Insertion, and Deletion

Node Creation

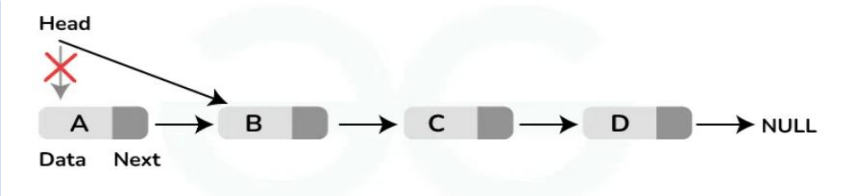
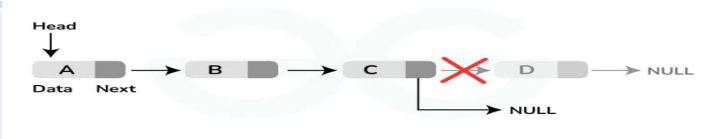
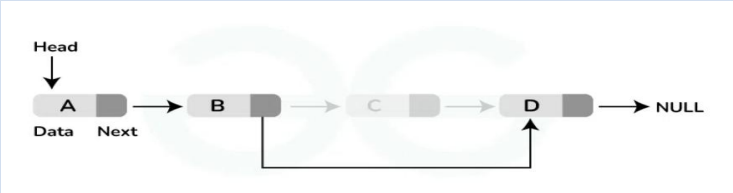
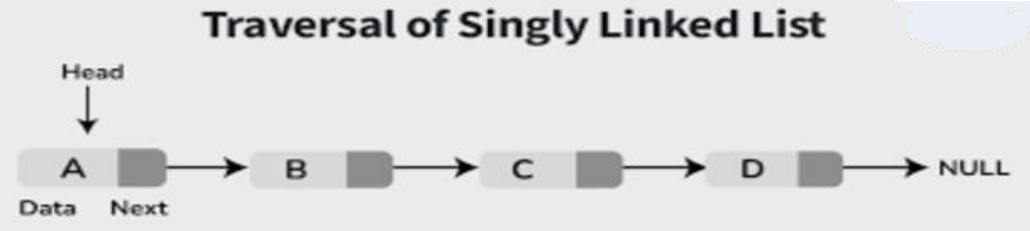
```
struct node
{
   int data;
   struct node *next;
};
struct node *head=NULL, *ptr;
ptr = (struct node *)malloc(sizeof(struct node));
```

A list of all such operations is given below.

Singly Linked List (Chains) : Operations

SN	Operation		Description
1	Insertion at beginning		It involves inserting any element at the front of the list. We just need to a few link adjustments to make the new node as the head of the list.
2	Insertion at end of the list		It involves insertion at the last of the linked list.
3	Insertion after specified node		It involves insertion after the specified node of the linked list. We need to skip the desired number of nodes in order to reach the node after which the new node will be inserted.

Singly Linked List (Chains) : Operations

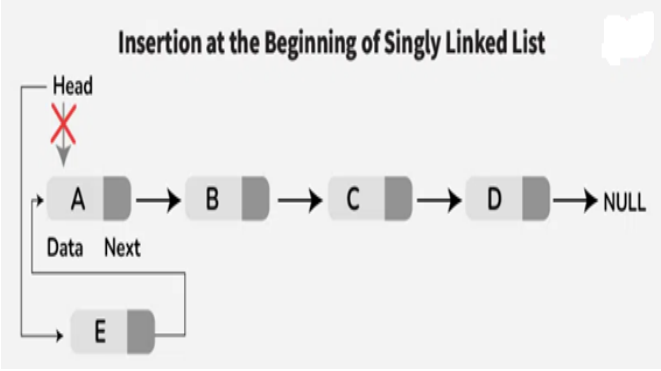
SN	Operation	Description
4	Deletion at beginning 	It involves deletion of a node from the beginning of the list. It just need a few adjustments in the node pointers.
5	Deletion at the end of the list 	It involves deleting the last node of the list.
6	Deletion after specified node 	It involves deleting the node after the specified node in the list. We need to skip the desired number of nodes to reach the node after which the node will be deleted. This requires traversing through the list.
7	Traversing 	In traversing, we simply visit each node of the list at least once in order to perform some specific operation on it, for example, printing data part of each node present in the list.
8	Searching	In searching, we match each element of the list with the given element. If the element is found on any of the location then location of that element is returned otherwise null is returned.

Linked List in C: Menu Driven Program

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
   int data;
   struct node *next;
};
struct node *head=NULL;
void begin_insert ();
void last_insert ();
void random_insert();
void begin_delete();
void last_delete();
void random_delete();
void display();
void search();
void main ()
{
   int choice;
   while(1)
   {
   printf("\n1.Insert in begining\n2.Insert at last\n3.Insert at random location\n4.Delete from Beginning");
   printf("\n5.Delete from last\n6.Delete node after specified location\n7.Search for an element\n8.Show\n9.Exit\n");
      printf("\nEnter your choice?\n");
      scanf("\n%d",&choice);
switch(choice)
{
         case 1:
         begin_insert();
         break;
         case 2:
         last_insert();
         break;
case 3:
         random_insert();
         break;
 case 4:
         begin_delete();
         break;
 case 5:
         last_delete();
         break;
 case 6:
         random_delete();
         break;
 case 7:
         search();
         break;
  case 8:
         display();
         break;
   case 9:
         free(head);
         exit(0);
    default:
         printf("Please enter valid choice..");
      }
   }
}
```

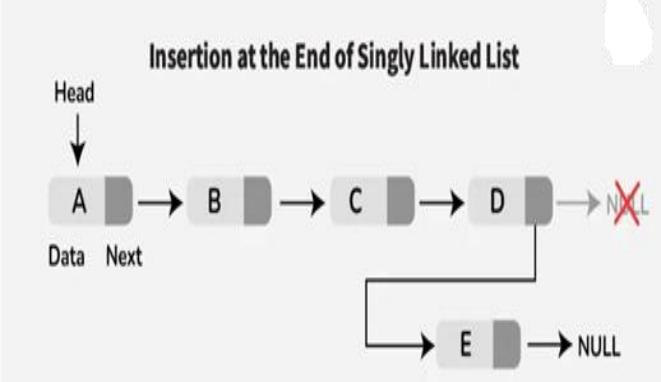
Linked List in C: Menu Driven Program

```
void begin_insert()
{
    struct node *ptr;
    int item;
    ptr = (struct node *) malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
else
    {
        printf("\nEnter value\n");
        scanf("%d",&item);
        ptr->data = item;
        if(head == NULL)
        {
            ptr -> next = NULL;
            head = ptr;
            printf("\nNode inserted");
        }
        else
        {
        ptr->next = head;
        head = ptr;
        printf("\nNode inserted");
    }
  }
}
```



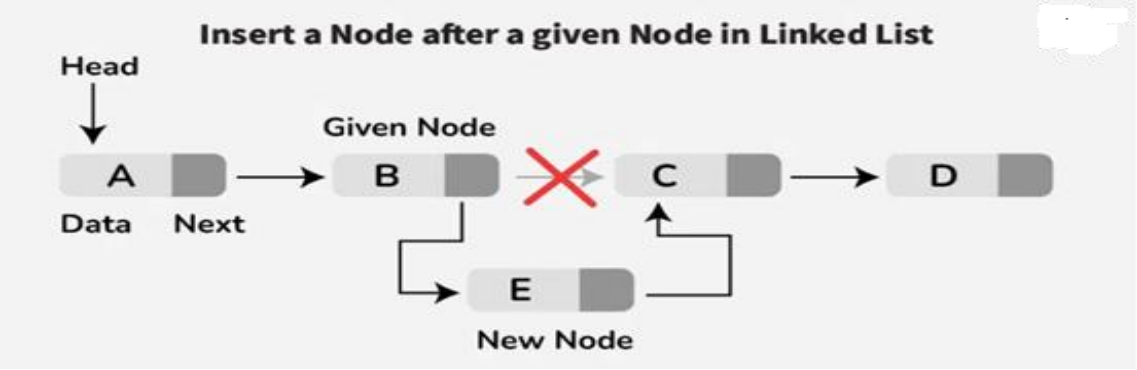
Linked List in C: Menu Driven Program

```
void last_insert()
{
    struct node *ptr,*temp;
    int item;
    ptr = (struct node*)malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
    else
    {
        printf("\nEnter value?\n");
        scanf("%d",&item);
        ptr->data = item;
        if(head == NULL)
        {
            ptr -> next = NULL;
            head = ptr;
            printf("\nNode inserted");
        }
        else
        {
            temp = head;
            while (temp -> next != NULL)
            {
                temp = temp -> next;
            }
            temp->next = ptr;
            ptr->next = NULL;
            printf("\nNode inserted");

        }
    }
}
```



Linked List in C: Menu Driven Program

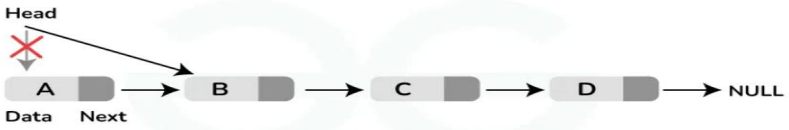
```
void random_insert()
{
    int i,loc,item;
    struct node *ptr, *temp;
    ptr = (struct node *) malloc (sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
```



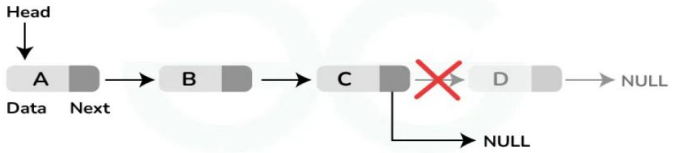

```
else
    {
      printf("\nEnter element value");
       scanf("%d",&item);
       ptr->data = item;
        printf("\nEnter the location  after which you want to insert ");
       scanf("%d",&loc);
        if(head==NULL)
        {
                printf("\nlist is empty, can't insert");
                return;
        }
       temp=head;
       for(i=1;i<loc;i++)
       {
          temp = temp->next;
          if(temp == NULL)
          {
             printf("\ncan't insert\n");
             return;
          }
       }
       ptr ->next = temp ->next;
       temp ->next = ptr;
       printf("\nNode inserted");
    }
}
```

Linked List in C: Menu Driven Program

```
void begin_delete()
{
   struct node *temp;
   if(head == NULL)
   {
      printf("\nList is empty\n");
   }
   else if(head -> next == NULL)
   {
      temp=head;
      head = NULL;
      free(temp);
      printf("\n The Only node of the list deleted ...\n");
   }
   else
   {
      temp = head;
      head = head->next;
      free(temp);
      printf("\nNode deleted from the begining ...\n");
   }
}
```

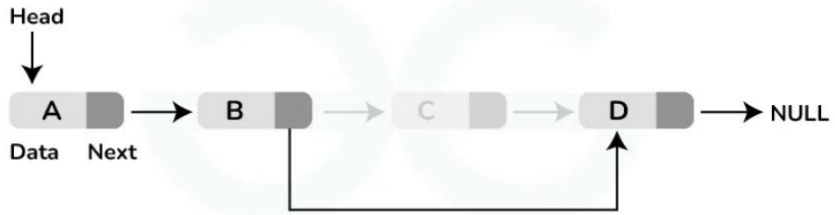



```
void last_delete()
{
   struct node *temp,*prev;
   if(head == NULL)
   {
      printf("\nlist is empty");
   }
   else if(head -> next == NULL)
   {
      temp=head;
      head = NULL;
      free(temp);
      printf("\n The Only node of the list deleted ...\n");
   }
   else
   {
      temp = head;
      while(temp->next != NULL)
      {
         prev = temp;
         temp = temp ->next;
      }
      prev->next = NULL;
      free(temp);
      printf("\nDeleted Node from the last ...\n");
   }
}
```



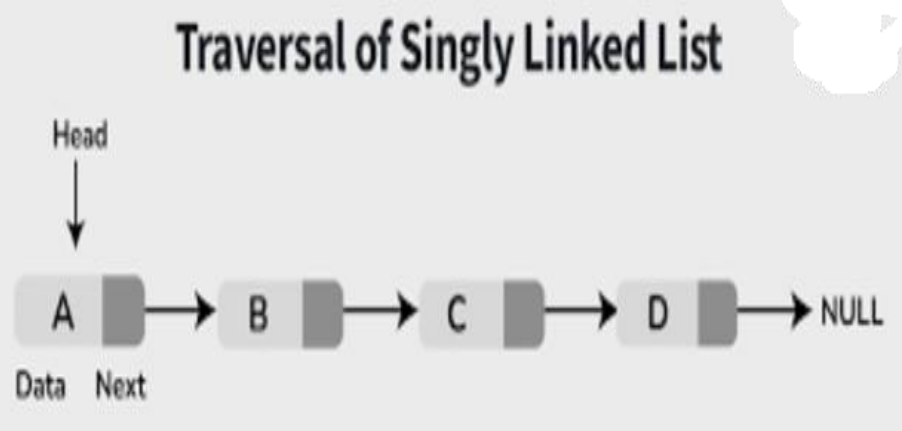
Linked List in C: Menu Driven Program

```
void random_delete()
{
   struct node *temp,*prev;
   int loc,i;
   if(head==NULL)
   {
        printf("\nlist is empty, can't delete");
      }
      else
      {
   printf("\n Enter the location of the node after which you want to perform deletion \n");
   scanf("%d",&loc);
   temp=head;
   for(i=1;i<=loc;i++)
    {
         prev = temp;
         temp = temp->next;
         if(temp == NULL)
         {
            printf("\nCan't delete");
            return;
         }
      }
      prev ->next = temp ->next;
      free(temp);
      printf("\nDeleted node %d ",loc+1);
   }
   }
```



Linked List in C: Menu Driven Program

```
void display()
{
    int count=0;
    struct node *temp=head;
    if(head == NULL)
    {
        printf("Nothing to print");
    }
    else
    {
        printf("\nprinting values . . . . .\n");
        while (temp!=NULL)
        {
            count++;
            printf("\n%d",temp->data);
            temp = temp -> next;
        }
    printf("\n The number of nodes %d",count);
  }
}
```



Linked List in C: Menu Driven Program

```
void search()
{
   struct node *temp=head;
   int key,i=1;
   if(head == NULL)
   {
      printf("\nEmpty List\n");
      return;
   }
   printf("\nEnter item which you want to search?\n");
   scanf("%d",&key);
   if(head->data == key)
   {
         printf("item found at location %d ",i);
    }
   else
    {
         while (temp!=NULL && temp->data!=key)
         {
            i++;
            temp = temp -> next;
         }
         if(temp==NULL)
         {
            printf("Item not found\n");
            return;
         }
         printf("item found at location %d ",i);
     }
}
```

Lab Program

Title: Design, Develop and Implement a menu driven Program in C for handling change of branch.

Program:

```
#include<stdio.h>
#include<stdlib.h>
struct SLL
{
        int usn;
        char name[20];
        char mode[20];
        struct SLL *next;
};
typedef struct SLL node;
node *start=NULL;
```

Lab Program

```
node* createNode()
{
        node *newnode;
        newnode=(node*)malloc(sizeof(node));
        printf("Enter the usn: \n");
        scanf("%d",&newnode->usn);
        printf("Enter the Name: \n");
        scanf("%s",newnode->name);
        printf("Enter the mode (Regular/Lateral/COB/COC): \n");
        scanf("%s",newnode->mode);
        newnode->next=NULL;
        return newnode;
}
```

```
void insertend()
{
        node *newnode,*temp;
        newnode=createNode();
        if(start==NULL)
        {
                start=newnode;
                return;
        }
        temp=start;
        while(temp->next!=NULL)
                temp=temp->next;
        temp->next=newnode;
}
```

Lab Program

```
void display()
{
        node* temp=start;
        if(start==NULL)
        {
                printf("list is empty\n");
                return;
        }
        printf("Students in the class are:\n");
        printf("USN\tNAME\tMode\n");
        while(temp!=NULL)
        {
        printf("%d\t%s\t%s\n",temp->usn,temp->name,temp->mode);
        temp=temp->next;
        }
        printf("\n");
}
```

Lab Program

```
void deletekey()
{
    node *temp=start,*prev;
    int key;
    if(start==NULL)
    {
            printf("List is empty. Deletion is not possible.\n");
            return;
    }
    printf("Enter the USN to be deleted: \n");
    scanf("%d",&key);
    if (start->usn == key)
    {
            start = start->next;
            printf("Deleted USN: %d\n",temp->usn);
            free(temp);
    }
else
{
        while(temp!=NULL && temp->usn!=key)
        {
                prev=temp;
                temp=temp->next;
        }
        if(temp==NULL)
        {
                printf("USN is not found, so can't delete.\n");
                return;
        }
        prev->next=temp->next;
        printf("Deleted USN: %d\n",temp->usn);
        free(temp);
}
}
```

Lab Program

```
int main()
{
int choice,n,i;
while(1)
{
printf("\nEnter your choice:\n");
printf("1 for insert\n");
printf("2 for display\n");
printf("3 for delete students based on COC/COB\n");
printf("4 for adding students from lateral entry/COC/COB\n");
printf("5 for exit\n");
scanf("%d",&choice);
switch(choice)
{
case 1:
case 4: printf("Enter the number of students\n");
        scanf("%d",&n);
        for(i=0;i<n;i++)
            insertend();
        break;
case 2: display();
        break;
case 3: deletekey();
        break;
case 5:
            free(start);
            exit(0);
default: printf("Invalid chioce\n");
}
}
return 0;
}
```